

ProviewR

OPEN SOURCE PROCESS CONTROL



I/O System handbok

2009-01-09

Copyright SSAB EMEA AB 2005-2024

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

Innehåll

Om den här handledningen	7
Introduktion.....	8
Översikt.....	9
Nivåer.....	9
Konfigurering.....	9
I/O System.....	10
PSS9000.....	11
Rack objekt.....	11
Rack_SSAB.....	11
Attribut.....	11
Drivrutin.....	11
Ssab_RemoteRack.....	11
Attribut.....	11
Di kort.....	12
Ssab_BaseDiCard.....	12
Ssab_DI32D.....	12
Do kort.....	12
Ssab_BaseDoCard.....	12
Ssab_DO32KTS.....	13
Ssab_DO32KTS_Stall.....	13
Ai kort.....	13
Ssab_BaseACard.....	13
Ssab_AI8uP.....	13
Ssab_AI16uP.....	13
Ssab_AI32uP.....	14
Ssab_AI16uP_Logger.....	14
Ao kort.....	14
Ssab_AO16uP.....	14
Ssab_AO8uP.....	14
Ssab_AO8uPL.....	14
Co kort.....	14
Ssab_CO4uP.....	14
Profibus.....	16
Profibus konfiguratorn.....	16
Address.....	17
SlaveGsdData.....	17
UserPrmData.....	17
Module.....	18
Specificera dataarean.....	19
Digitala ingångar.....	20
Analoga ingångar.....	20
Digitala utgångar.....	20
Analoga utgångar.....	20
Komplexa dataareor.....	20
Drivrutin.....	20
Agent objekt.....	20
Pb_Profiboard.....	20
Pb_Hilscher.....	21
Slavobjekt.....	21
Pb_Dp_Slave.....	21
ABB_ACS_Pb_Slave.....	21
Siemens_ET200S_IM151.....	21
Siemens ET200M_IM153.....	21
Modulobjekt.....	21
Pb_Module.....	21
ABB_ACS_PPO5.....	21

Siemens_ET200S_Ai2.....	21
Siemens_ET200S_Ao2.....	21
Siemens_ET200M_Di4.....	21
Siemens_ET200M_Di2.....	22
Siemens_ET200M_Do4.....	22
Siemens_ET200M_Do2.....	22
Konfiguration av Hilscher kort.....	22
MotionControl USB I/O.....	22
Drivrutin.....	22
Rackobjekt.....	23
MotonControl_USB.....	23
Kortobjekt.....	23
MotionControl_USBIO.....	23
Kanaler.....	23
Ai konfigurering.....	24
Ao konfigurering.....	24
Länkfil.....	24
Velleman K8055.....	25
Agentobjekt.....	25
USB_Agent.....	25
Rackobjekt.....	25
Velleman_K8055.....	25
Kortobjekt.....	25
Velleman_K8055_Board.....	25
Kanaler.....	26
Ai konfigurering.....	26
Ao konfigurering.....	26
Länkfil.....	26
Adaption av I/O system.....	28
Översikt.....	28
Nivåer.....	28
Area objekt.....	29
I/O objekt.....	29
Processer.....	29
Ramverk.....	29
Metoder.....	30
Ramverk.....	30
Skapa I/O-objekt.....	32
Flags.....	34
Attribut.....	35
Description.....	35
Process.....	35
ThreadObject.....	35
Metod objekt.....	35
Agenter.....	36
Rack.....	36
Kort.....	36
Connect-metod för ThreadObject.....	36
Metoder.....	37
Lokal datastruktur.....	37
Agent-Metoder.....	37
IoAgentInit.....	37
IoAgentClose.....	38
IoAgentRead.....	38
IoAgentWrite.....	38
IoAgentSwap.....	38
Rack-metoder.....	38
IoRackInit.....	38
IoRackClose.....	38
IoRackRead.....	38

IoRackWrite.....	38
IoRackSwap.....	38
Card-metoder.....	39
IoCardInit.....	39
IoCardClose.....	39
IoCardRead.....	39
IoCardWrite.....	39
IoCardSwap.....	39
Registrering av metoder.....	39
Registrering av klassen.....	39
Modul i ProviewR's bassystem.....	39
Projekt.....	40
Exempel på rack metoder.....	40
Exempel på metoder digitalt ingångskort.....	41
Exempel på metoder för digitalt utgångskort.....	44
Steg för steg beskrivning.....	47
Lägga in i ett projekt.....	47
Skapa klasser.....	47
Skapa en klassvolym.....	47
Öppna klassvolymen.....	48
Skapa en Rack klass.....	49
Skapa en kortklass.....	50
Bygg klassvolymen.....	56
Installera drivrutin.....	56
Skriva metoder.....	57
Registrera klassen.....	61
Makefile.....	61
Länkfil.....	62
Konfigurera nodhierarkin.....	62
Lägga in i ProviewR's bassystem.....	65
Klassvolym.....	65
OtherIO.....	65
Skapa klasser.....	66
Skapa metoder.....	66
Registrera klassen.....	66
Objektbilder.....	66
Merge.....	67
Separat Modul.....	67
Om Hilscher Profibus agenten.....	67

Om den här handledningen

ProviewR's *I/O System handbok* är avsedd för personer som vill knyta olika typer av I/O system till ProviewR, och för användare som vill ha en djupare förståelse för hur I/O hanteringen i ProviewR fungerar.

Introduktion

IO-hanteringen i ProviewR består av ett ramverk som är designat för att

- vara portabelt och körbart på olika plattformar.
- enkelt kunna lägga till nya I/O-system.
- hantera I/O-kort på den lokala bussen.
- hantera distribuerade I/O-system och kommunicera med remota rack-system.
- tillåta projekt att implementera lokala I/O system.
- synkronisera I/O-system med exekveringen av plc-program, eller med applikationsprocesser.

Översikt

I/O på en processnod konfigureras genom att skapa objekt i ProviewR databasen. Objekten är uppdelade i två träd, anläggningshierarkin och nodhierarkin. Anläggningshierarkin beskriver hur anläggningen är uppbyggd med olika processavsnitt, motorer, pumpar, fläktar mm. Här återfinns signalobjekt som representerar värden som läses in från olika givare, eller värden som ställs ut till motorer, ställdon mm. Signalobjekten är av klasserna Di, Do, Ai, Ao, Ii, Io, Co eller Po. Nodhierarkin beskriver processdatorns uppbyggnad, med server processer och I/O system. I/O systemet konfigureras med ett träd av agent, rack, kort och kanal-objekt. Kanalobjekten representerar en I/O signal som kommer in till datorn via en kanal på ett I/O kort (eller via ett distribuerat bussystem). Kanalobjekten är av klasserna ChanDi, ChanDo, ChanAi, ChanAo, ChanIi, ChanIo och ChanCo. Varje signalobjekt i anläggningshierarkin pekar på ett kanalobjekt i nodhierarkin. Kopplingen motsvarar den fysiska anslutningen mellan en givare och kanalen på ett I/O kort.

Nivåer

I/O objekten för en processnod konfigureras i en trädstruktur med tre nivåer: Agent, Rack och Kort. I vissa fall kan även en fjärde nivå närvara, Kanaler. Kanal objekten kan konfigureras som individuella objekt eller ligga som interna attribut i Kort objektet.

Konfigurering

För ett I/O-system på den lokala bussen används ofta endast rack och kort-nivån. En konfigurering kan gå till så här. Ett rackobjekt läggs under \$Node-objektet, och under detta ett kortobjekt för varje I/O kort som finns i racken. Kortobjektet innehåller kanalobjekt för de kanaler som finns på respektive kort. Kanalobjekten kopplas till signal-objekt i anläggningshierarkin. Kanalerna för analoga signaler innehåller attribut för att ange mätområden, och kortobjekten innehåller attribut för adresser.

Konfigureringen av ett distribuerat I/O-system kan se lite annorlunda ut. Fortfarande används nivåerna Agent, Rack, Kort och Kanal, men nivåerna får en annan innebörd. Om vi tar Profibus som exempel, utgörs agentnivån av ett objekt för masterkort som är monterat på datorn. Racknivån utgörs av slavobjekt, som representerar profibus-slavar som sitter inkopplade på profibus-slingan. Kortnivån utgörs av modulobjekt som representerar moduler som hanteras av slavar. Kanalobjekten representerar data som skickas på bussen från masterkortet ut till modulerna eller vv.

I/O System

Här följer en beskrivning på de I/O system som är implementerade i ProviewR.

PSS9000

PSS9000 består av I/O kort för analoga in, analoga ut, digitala ut och digitala in. Det finns även kort för pulsräkning och PID reglering. Korten sätts i ett rack med bussen QBUS, en buss ursprungligen utvecklad för Digitals PDP-11 processor. Racken kopplas via en PCI-QBUS konverterare till en x86-pc, eller kopplas via Ethernet, s k remoterack.

Systemet konfigureras med objekt som ligger i SsabOx volymen. Här finns objekt som representerar rack- och kort-nivån. Agent-nivån representeras av \$Node objektet.

Rack objekt

Rack_SSAB

Rack_SSAB objektet representerar ett 19" PSS9000 ramverk med QBUS som bakplan. Antalet kortplatser kan variera.

Ramverket kopplas till en x86 pc med en PCI-QBUS konverteringskort, PCI-Q, som sätts i pc'n och ansluts till ramverket med kabel. Flera rack kan anslutas via bussförlängarkort.

Rackobjekten placeras under \$Node objektet och namnges C1, C2, osv (i äldre system förekommer namnstandarden R1, R1 osv).

Attribut

Rack_SSAB innehåller inte några attribut används av systemet.

Drivrutin

PCI-QBUS konverteringskortet, PCI-Q, kräver att en drivrutin installeras.

Ssab_RemoteRack

Ssab_RemoteRack objektet konfigurerar ett PSS9000 ramverk som ansluts till pc'n via Ethernet. Anslutningen av ramverket till Ethernet sker genom ett BFBETH kort sätts i ramverket.

Objektet placeras under \$Node objektet och namnges E1, E2 osv.

Attribut

<i>Attribut</i>	<i>Beskrivning</i>
Address	ip-adress för BTBETH kortet.
LocalPort	Port i processtationen.
RemotePort	Port för BTBETH kortet. Default 8000.
Process	Process som ska hantera racken. 1 plcprogrammet, 2 io_comm.
ThreadObject	Om process är 1, anges här den tråd i plcprogrammet som ska hantera racken.
StallAction	No, ResetInputs eller EmergencyBreak. Default EmergencyBreak.

Di kort

Samtliga digitala ingångskort har en gemensam basklass, Ssab_BaseDiCard, som innehåller attribut som är gemensamma för alla di-kort. Objekten för respektive korttyp är utökade med kanalobjekt för de kanaler som kortet innehåller.

Ssab_BaseDiCard

<i>Attribut</i>	<i>Beskrivning</i>
RegAddress	QBUS adress för kortet.
ErrorHardLimit	Felgräns som stoppar systemet.
ErrorSoftLimit	Felgräns som larmar.
Process	Process som ska hantera racken. 1 plcprogrammet, 2 io_comm.
ThreadObject	Om process är 1, anges här den tråd i plcprogrammet som ska hantera racken.
ConvMask1	Konverteringsmasken anger vilka kanaler som ska omvandlas till signalvärden. Hanterar kanal 1 – 16.
ConvMask2	Se ConvMask1. Hanterar kanal 17 – 32.
InvMask1	Inverteringsmasken anger vilka kanaler som är inverterade. Hanterar kanal 1-16.
InvMask2	Se InvMask1. Hanterar kanal 17 – 32.

Ssab_DI32D

Objekt som konfigurerar ett digitalt ingångskort av typen DI32D. Kortet innehåller 32 kanaler, vars DiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseDiCard ovan.

Do kort

Samtliga digitala utgångskort har en gemensam basklass, Ssab_BaseDoCard, som innehåller attribut som är gemensamma för alla do-kort. Objekten för respektive korttyp är utökade med kanalobjekt för de kanaler som kortet innehåller.

Ssab_BaseDoCard

<i>Attribut</i>	<i>Beskrivning</i>
RegAddress	QBUS adress för kortet.
ErrorHardLimit	Felgräns som stoppar systemet.
ErrorSoftLimit	Felgräns som larmar.
Process	Process som ska hantera racken. 1 plcprogrammet, 2 io_comm.
ThreadObject	Om process är 1, anges här den tråd i plcprogrammet som ska hantera kortet.
InvMask1	Inverteringsmasken anger vilka kanaler som är inverterade. Hanterar kanal 1-16.
InvMask2	Se InvMask1. Hanterar kanal 17 – 32.
FixedOutValue1	Bitmask för kanal 1 to 16 vid nödstopp av I/O hanteringen. FixedOutValue ska normalt vara 0, eftersom detta är värdet vid spänningsbortfall.
FixedOutValue2	SeFixedOutValue1. FixedOutValue2 är bitmask för kanal 17 – 32.
ConvMask1	Konverteringsmasken anger vilka kanaler som ska omvandlas till signalvärden. Hanterar kanal 1 – 16.
ConvMask2	Se ConvMask1. Hanterar kanal 17 – 32.

Ssab_DO32KTS

Objekt som konfigurerar ett digitalt utgångskort av typen DO32KTS. Kortet innehåller 32 kanaler, vars DoChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseDoCard ovan.

Ssab_DO32KTS_Stall

Objekt som konfigurerar ett digitalt utgångskort av typen DO32KTS Stall. Kortet liknar DO32KTS men innehåller även en stall-funktion. som gör reset på bussen, dvs alla utgångar nollställs på samtliga kort, om ingen skrivning eller läsning har gjort på kortet inom ca 1.5 sekunder.

Ai kort

Samtliga analoga kort har en gemensam basklass, Ssab_BaseACard, som innehåller attribut som är gemensamma för alla analoga kort. Objektet för respektive korttyp är utökade med kanalobjekt för de kanaler som kortet innehåller.

Ssab_BaseACard

<i>Attribut</i>	<i>Beskrivning</i>
RegAddress	QBUS adress för kortet.
ErrorHardLimit	Felgräns som stoppar systemet.
ErrorSoftLimit	Felgräns som larmar.
Process	Process som ska hantera racken. 1 plcprogrammet, 2 io_comm.
ThreadObject	Om process är 1, anges här den tråd i plcprogrammet som ska hantera kortet.

Ssab_AI8uP

Objekt som konfigurerar ett analogt ingångskort av typen Ai8uP. Kortet innehåller 8 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ssab_AI16uP

Objekt som konfigurerar ett analogt ingångskort av typen Ai16uP. Kortet innehåller 16 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ssab_AI32uP

Objekt som konfigurerar ett analogt ingångskort av typen Ai32uP. Kortet innehåller 32 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ssab_AI16uP_Logger

Objekt som konfigurerar ett analogt ingångskort av typen Ai16uP_Logger. Kortet innehåller 16 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ao kort

Ssab_AO16uP

Objekt som konfigurerar ett analogt ingångskort av typen AO16uP. Kortet innehåller 16 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ssab_AO8uP

Objekt som konfigurerar ett analogt ingångskort av typen AO8uP. Kortet innehåller 8 kanaler, vars

AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Ssab_AO8uPL

Objekt som konfigurerar ett analogt ingångkort av typen AO8uPL. Kortet innehåller 8 kanaler, vars AiChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt. Attribut: se BaseACard ovan.

Co kort

Ssab_CO4uP

Objekt som konfigurerar ett pulsräknarkort av typen CO4uP. Kortet innehåller 4 kanaler, vars CoChan objekt ligger internt i objektet. Objektet läggs som barn till ett Rack_SSAB eller Ssab_RemoteRack objekt.

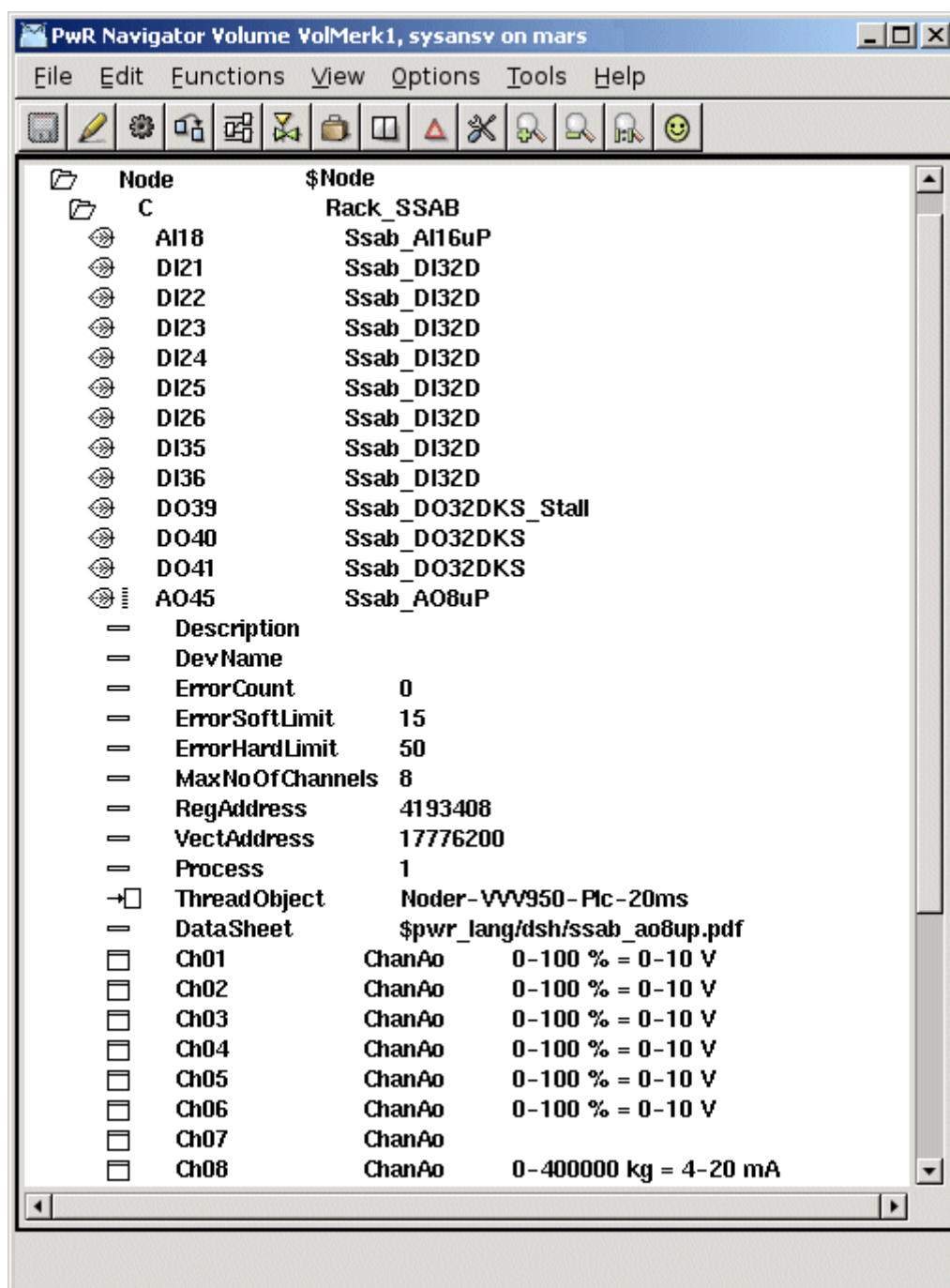


Fig Exempel på PSS9000 konfiguration

Profibus

Profibus är en fältbuss med noder av typen master eller slav. Det vanligaste är monomastersystem med en master och upp till 125 slavar. Varje slav kan innehålla en eller flera moduler.

I ProviewR's I/O hantering representerar mastern agent-nivån, slavarerna rack-nivån och modulerna kort-nivån.

ProviewR har för närvarande stöd för två olika masterkort, Softing PROFIBoard PCI (se www.softing.com) och Hilscher CIF 50-PB (se www.hilscher.com). Masterkortet ansluts till processnodens PCI-buss. Kortet konfigureras med ett objektet av klassen Profibus:Pb_Profiboard som läggs under \$Node objektet.

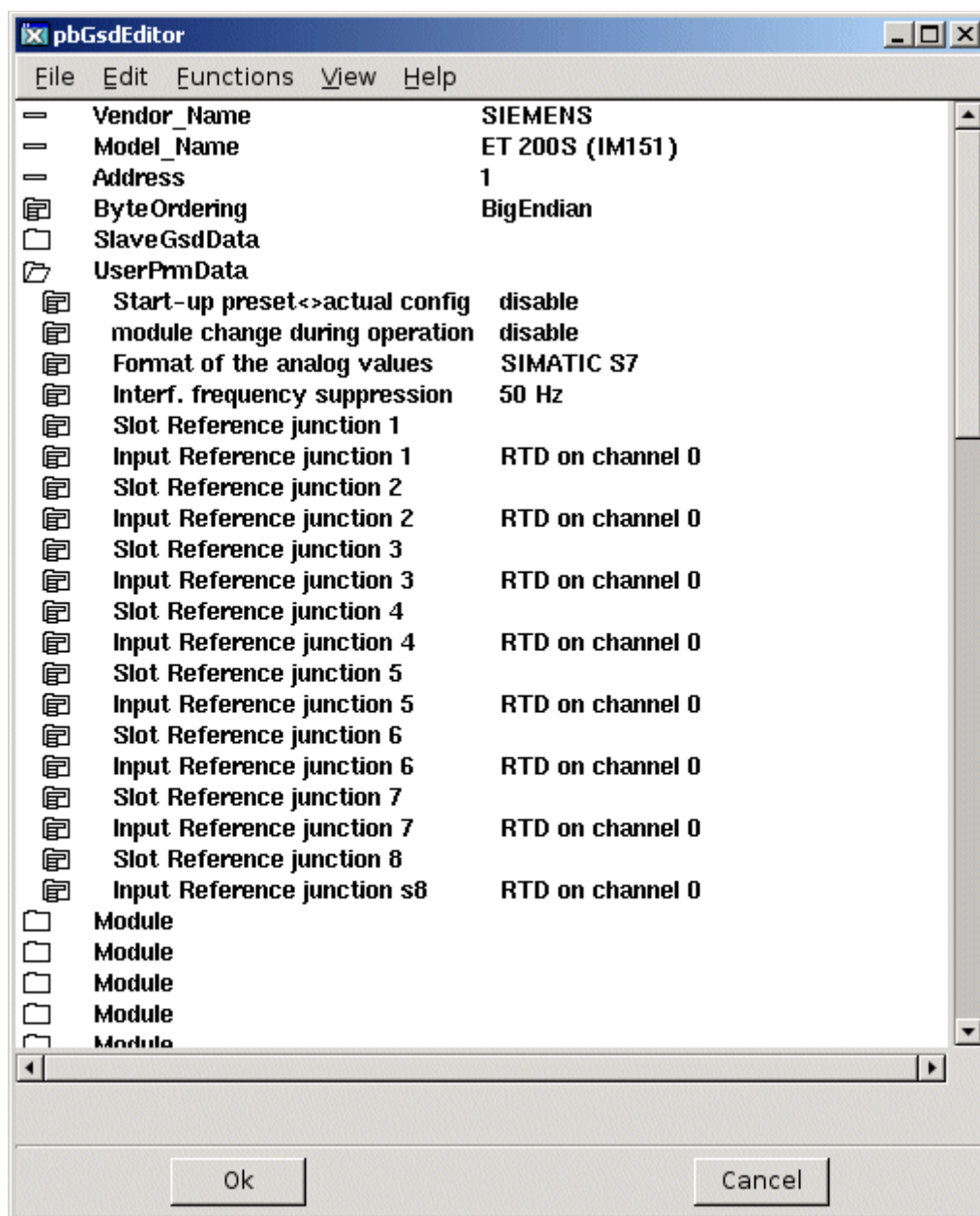
Varje slav som är kopplad till profibus-slingan konfigureras med objekt av klassen Pb_DP_Slave eller subclasser till denna klass. Slavobjekten läggs som barn till master-objektet. För slavobjektet kan man öppna en speciell profibus-konfigurator, som konfigurerar slavobjektet, och skapar modulobjekt för de moduler som är kopplade till slaven. Profibus-konfiguratorn utgår från gsd-filen för slaven. gsd-filen är en textfil som tillhandahålls av leverantören, och beskriver de olika konfigurationer som kan göras för den aktuella slaven. Innan man öppnar profibus-konfiguratorn måste man ange vilken gsd-fil som den ska utgå ifrån. Det här gör man genom att lägga gsd-filen för den aktuella slaven på \$pwrp_exe, och lägga in filnamnet i attributet GSDfile i slavobjektet.

Om det finns en subclass för den slav man ska konfigurera, t ex, Siemens_ET200S_IM151, finns normalt gsd-filen angiven i slavobjektet och gsd-filen följer med ProviewR distributionen.

När den här operationen är utförd, kan man öppna profibus konfiguratorn genom att högerklicka på objektet och aktivera 'ConfigureSlave' i popupmenyn.

Profibus konfiguratorn

Profibus konfiguratorn kan öppnas för ett slavobjekt, dvs ett objekt av klassen Pb_DP_Slave eller av en subclass av denna. Förutsättningen är att det finns en läsbar gsd-fil angiven i GSDfile attributet.



Address

Slavens adress anges i Address attributet. Adressen har ett värde i intervallet 0-125 som vanligen anges med omkopplare på slaven.

SlaveGsdData

Mappen SlaveGsdData innehåller diverse data av informativ karaktär.

UserPrmData

Mappen SlaveGsdData innehåller de parametrar som kan konfigureras för den aktuella slaven.

Module

En slav har plats för en eller flera moduler. Det finns modulära slavar med enbart en modul, där slaven och moduler utgör en enhet, och det finns slavar av rack typ som man kan hänga på ett stort

antal moduler. Profibuskonfiguratorn visar en mapp för varje modul som kan konfigureras för slaven.

För varje modul anges ett objektsnamn, t ex M1, M2 etc. Moduler på samma slav måste ha olika objektsnamn.

Vidare anges även modultypen. Denna väljs från en lista av modultyper som supportas av den aktuella slaven. Listan ligger under 'Type'.

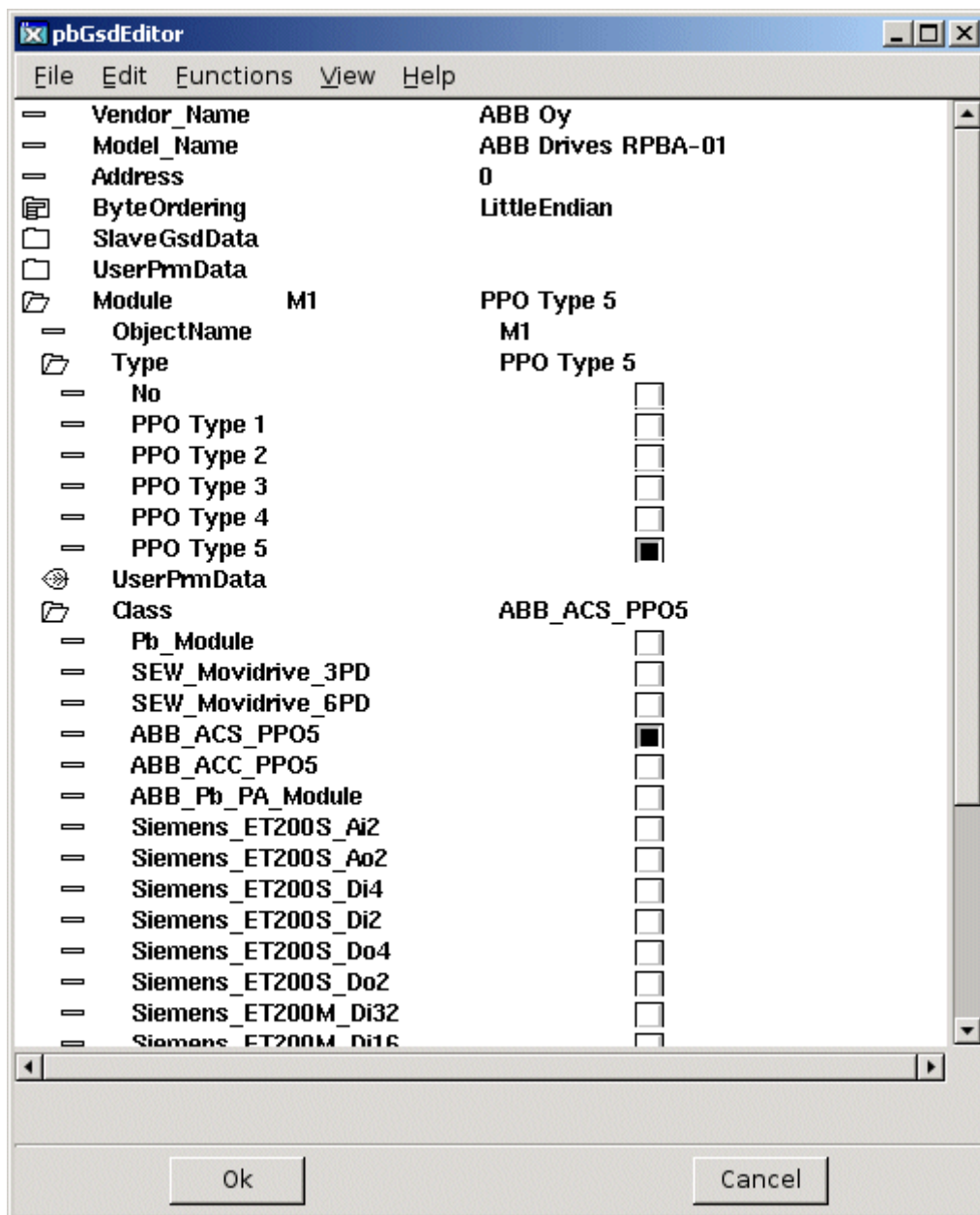


Fig Module Type and Class selected

När man har valt typ konfigureras parametrar för den valda modultypen under UserPrmData.

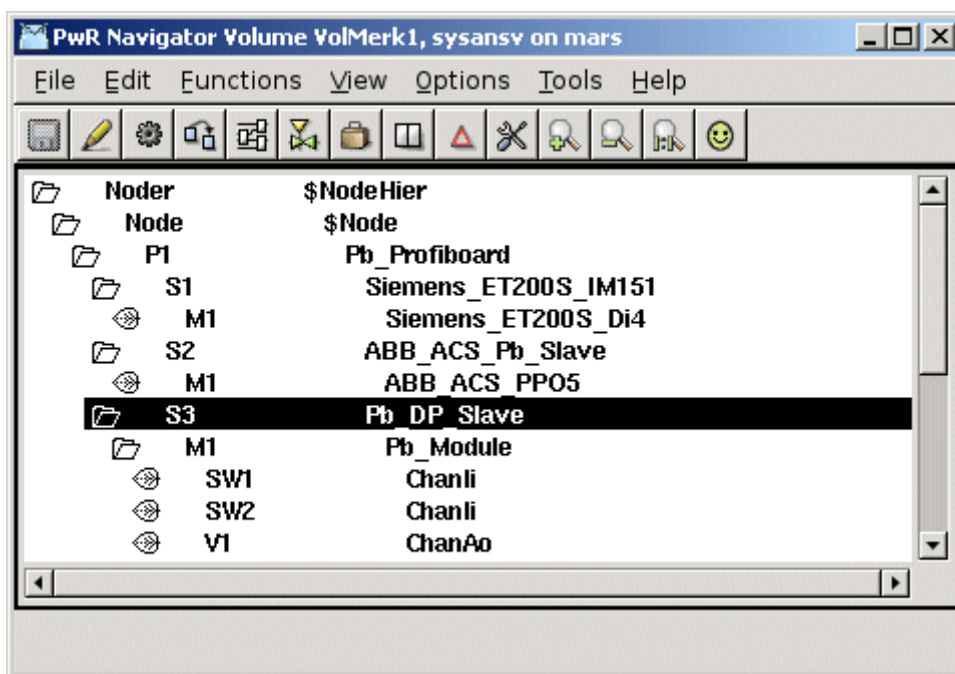
Man måste även ange en klass för modul-objektet. Vid konfigureringen skapas ett modulobjekt för varje konfigurerad modul. Objektet är av klassen Pb_Module eller en subclass av denna, och under 'Class' listas alla subclasser till Pb_Module. Hittar man någon som motsvarar aktuell modultyp anger man denna, annars väljer man basklassen Pb_Module. Skillnaden mellan subclasserna och basklassen är att det i subclasserna finns en definierad dataarea i form av kanalobjekt (se avsnitt Specifiera Dataarean nedan).

När alla moduler är konfigurerade sparar man genom att klicka på 'Ok' och går ur genom att klicka på 'Cancel'. Modulobjekt med angivet objektsnamn och angiven klass har nu skapats under slav-objektet.

Förhoppningsvis hittar man ett modulobjekt som motsvarar den aktuella modulen. Kriteriet för om ett modulobjekt är användbar eller inte, är om specifikationen av dataarean matchar den aktuella modulen. Om man inte hittar ett lämpligt modulobjekt finns två möjligheter: att i en skapa en ny klass med Pb_Module som basclass, och i denna lägga in lämpliga kanalobjekt, eller att konfigurera kanalobjekten under ett Pb_Module objekt. Det senaste alternativet är enklast om det är frågan om enstaka modulobjekt, men har man många moduler av samma typ bör man överväga att skapa en klass för denna.

Specificera dataarean

Nästa steg är att specificera dataarean för en modul. Ingångsmoduler läser in data som skickas till processnoden på bussen, och utgångsmoduler tar emot data som ställs ut. Det finns även moduler som båda tar emot och ställer ut data, t ex frekvensomformare. Hur dataarean som tas emot resp skickas på bussen måste konfigureras, och det görs med kanalobjekt. Inarean specificeras med ChanDi, ChanAi och ChanLi objekt, och utarean med ChanDo, ChanAo och ChanIo objekt. Kanalobjekten läggs som barn till modulobjektet, eller, om man väljer att göra en speciell klass för modulen, som interna attribut i modulobjektet. I kanalobjektet måste man ange Representation, som specificerar formatet på en variabel, och i vissa fall även Number (för Bit-representation). I slavobjektet kan man även behöva ange ByteOrdering (LittleEndian eller BigEndian) och FloatRepresentation (Intel eller IEEE).



Digitala ingångar

Digitala ingångsmoduler skickar ingångarna som bitar i ett ord. Varje ingång specificeras med ett ChanDi objekt. Representation sätts till Bit8, Bit16, Bit32 eller Bit64, beroende på ordets storlek, och Number anger det bit-nummer i ordet som innehåller kanalvärdet (första biten har nummer 0).

Analoga ingångar

En analog ingång överförs vanligtvis som ett heltal och specificeras med ett ChanAi objekt. Representation matchas mot heltalsformatet i överföringen. I vissa fall skickas värdet som ett flyttal, och då måste flyttalsformatet anges i slavobjektets FloatRepresentation (FloatIntel eller FloatIEEE). Område för konvertering till ingenjörstorhet specificeras i RawValueRange, ChannelSigValueRange, SensorSigValueRange och ActValRange (eftersom signalvärde inte används kan ChannelSigValRange och SensorSigValRange sättas till samma värde som RawValRange).

Digitala utgångar

Digitala utgångsmoduler specificeras med ChanDo objekt. Representation sätts till Bit8, Bit16, Bit32 eller Bit64 beroende på formatet vid överföringen.

Analoga utgångar

För analoga utgångar används ChanAo objekt. Sätt Representation och ange områden för konvertering från ingenjörstorhet till överförings område (sätt ChannelSigValRange och SensorSigValRange till samma värde som RawValRange).

Komplexa dataareor

Många moduler skickar en blandning av heltal, flyttal, bitmaskar etc. Man får då kombinera kanalobjekt av olika typ. Kanalobjekten ska ligga i samma ordning som det data de representerar är organiserade i dataarean. För moduler med både in- out utarea lägger man normalt inareans kanalerna först och därefter utareans kanalerna.

Drivrutin

Både Softing PROFiBoard och Hilscher CIF 50-PB kräver installation av en drivrutin. I båda fallen tillhandahålls drivrutinerna från respektive tillverkare: www.softing.com för Softing och www.hilscher.com för Hilscher.

Agent objekt

Pb_ProfiBoard

Agent objekt för en profibus master av typen Softing PROFiBoard. Objektet placeras i nodhierarkin under nodobjektet.

Pb_Hilscher

Agent objekt för en profibus master av typen Hilscher CIF 50-PB. Objektet placeras i nodhierarkin under nodobjektet.

Slavobjekt

Pb_Dp_Slave

Basobjekt för en profibus slav. Placeras under ett profibus agentobjekt. I GSDfile attributet anges en gsd-fil för den aktuella slaven. När gsd-filen är angiven kan slaven konfigurera med Profibuskonfiguratorn.

ABB_ACS_Pb_Slave

Slavobjekt för en frekvensomformare ABB ACS800 med protokoll PPO5.

Siemens_ET200S_IM151

Slavobjekt för en Siemens ET200S IM151.

Siemens ET200M_IM153

Slavobjekt för en Siemens ET200M IM153.

Modulobjekt

Pb_Module

Basklass för en profibus modul. Skapas av profibuskonfiguratorn. Placeras som barn till ett slavobjekt.

ABB_ACS_PPO5

Modulobjekt för en frekvensomformare ABB ACS800 med protokoll PPO5.

Siemens_ET200S_Ai2

Modulobjekt för en Siemens ET200S modul med 2 analoga ingångar.

Siemens_ET200S_Ao2

Modulobjekt för en Siemens ET200S modul med 2 analoga utgångar.

Siemens_ET200M_Di4

Modulobjekt för en Siemens ET200M modul med 4 digitala ingångar.

Siemens_ET200M_Di2

Modulobjekt för en Siemens ET200M modul med 2 digitala ingångar.

Siemens_ET200M_Do4

Modulobjekt för en Siemens ET200M modul med 4 digitalt utgångar.

Siemens_ET200M_Do2

Modulobjekt för en Siemens ET200M modul med 2 digitala utgångar.

Konfiguration av Hilscher kort

Hilscher CIF 50-PB korten är förinställda att konfigureras offline av Hilschers konfigurationsverktyg SyCon. Hilscher tillhandahåller dock instruktioner för hur denna konfigurationsdatabas ska tas bort, vilket medför att korten kan konfigureras online. Pb_Hilscher agenten innehåller kod för att radera konfigurationsdatabasen från flash, denna kod är dock normalt inte aktiverad. Genom att definiera makrot FLASH_WRITE_ENABLE i början av källkodsfilen rt_io_m_pb_hilscher.c och kompilera om denna aktiveras flashrutinerna. Därmed kommer SyCon databasen tas bort från kortet när agenten initieras. Flashningen är dock inte grundligt testad.

Det finns också ett verktyg vid namn cif50_rmdb som kan radera SyCon databasen från flash. Efter detta kan kortet användas i ProviewR.

MotionControl USB I/O

Motion Control USB I/O är ett kort tillverkas av Motion Control, www.motioncontrol.se. Kortet kopplas till USB porten på pc'n. På kortet finns 21 kanaler av olika typ, uppdelade på 3 portar, A, B och C. De första fyra kanalerna (A1 – A4) är Digitala utgångar av relätyp som tål upp till 230 V. Nästa fyra kanaler (A5 – A8) är Digitala utgångar med optokopplare. Nästa åtta kanaler (B1 – B8) kan antingen konfigureras som digitala utgångar, digitala ingångar eller analoga ingångar. Därefter följer fem kanaler (C1 – C5) som kan vara digitala ingångar eller utgångar, där den fjärde och femte även kan konfigureras som analoga utsignaler.

I ProviewR konfigureras USB I/O med ett rackobjekt, OtherIO:MotionControl_USB, som läggs i nodehierarkin under \$Node objekt, och ett kortobjekt, OtherIO:MotionContro_USBIO. Under kortobjektet läggs kanalobjekt av den typ som kortet är konfigurerat med.

Kortet har en watchdog som återställer kortets utgångar om man inte har skrivit till kortet inom en viss tid.

F n kan drivrutinen endast hantera ett kort.

Drivrutin

Ladda hem och packa upp tar-filen för drivrutinen.

```
> tar -xvzf usbio.tar.tz
```

Bygg drivrutinen mha make

```
> cd usbio/driver/linux-2.6
```

```
> make
```

Installera drivrutinen `usbio.ko` som root

```
> insmod usbio.ko
```

Tillåt alla att läsa och skriva till drivrutinen

```
> chmod a+rw /dev/usbio0
```

Till drivrutinen hör ett API med ett arkiv, `usbio/test/libusbio.a`. Kopiera arkivet till `/usr/lib` eller `$pwrp_lib` på utvecklingsnoden.

Rackobjekt

MotonControl_USB

Rackobjektet läggs under \$Node objektet i nodehierkin. Process ska vara 1. Koppla objektet till en plc-tråd genom att välja ut ett PlcThread-objekt och aktivera *Connect PlcThread* i poupmenyn för rackobjektet.

Kortobjekt

MotionControl_USBIO

Kortobjektet läggs under rackobjektet. Även här ska Process vara 1 och objektet ska kopplas till en plc-tråd. Ange kortets identitet, som finns angivet på kretskortet, i attributet Address. Watchdoggen aktiveras om man lägger in ett värde i WatchdogTime, som anger timeout-tiden i sekunder.

Kanaler

Kortets kanaler konfigureras under kortobjektet med kanalobjekt. Kanalobjektets Number attribut anger vilken kanal objektet konfigurerar (0-20), och objektets klass anger om kanalen används som en Di, Do, Ai eller Ao. Tabellen nedan visar hur kanalerna kan konfigureras.

<i>Channel</i>	<i>Type</i>	<i>Number</i>
A1	ChanDo	0
A2	ChanDo	1
A3	ChanDo	2
A4	ChanDo	3
A5	ChanDi	4

<i>Channel</i>	<i>Type</i>	<i>Number</i>
A6	ChanDi	5
A7	ChanDi	6
A8	ChanDi	7
B1	ChanDi, ChanDo or ChanAi	8
B2	ChanDi, ChanDo or ChanAi	9
B3	ChanDi, ChanDo or ChanAi	10
B4	ChanDi, ChanDo or ChanAi	11
B5	ChanDi, ChanDo or ChanAi	12
B6	ChanDi, ChanDo or ChanAi	13
B7	ChanDi, ChanDo or ChanAi	14
B8	ChanDi, ChanDo or ChanAi	15
C1	ChanDi or ChanDo	16
C2	ChanDi or ChanDo	17
C3	ChanDi or ChanDo	18
C4	ChanDi, ChanDo or ChanAo	19
C5	ChanDi, ChanDo or ChanAo	20

Ai konfiguration

Ai kanalerna har råvärdesområde 0 – 1023 och signalområde 0 – 5 V, dvs RawValRange och ChannelSigValRange ska sättas till

RawValRangeLow	0
RawValRangeHigh	1023
ChannelSigValRangeLow	0
ChannelSigValRangeHigh	5

Till exempel för att få ActualValue området 0 – 100 sätt SensorSigValRange 0 - 5 och ActValRange 0 – 100.

Ao konfiguration

Ao kanalerna har råvärdesområde 0 – 5 och signalområde 0 – 5 V, dvs RawValRange och ChannelSigValRange ska sättas till

RawValRangeLow	0
RawValRangeHigh	5
ChannelSigValRangeLow	0
ChannelSigValRangeHigh	5

Till exempel för att få ActualValue området 0 – 100 sätt SensorSigValRange 0 - 5 och ActValRange 0 – 100.

Länkfil

Arkivet med API't till drivrutinen måste länkas med plcprogrammet. Detta gör man genom att skapa filen \$pwrp_exe/plc_'nodnamn'_ 'busnr'.opt, t ex \$pwrp_exe/plc_mynode_0517.opt med

innehållet

```
$pwr_obj/rt_io_user.o -lpwr_rt -lusbio -lpwr_usb_dummy -lpwr_pnak_dummy
```

Velleman K8055

Velleman K8055 är ett USB kort med 2 Ai, 5 Di, 8 Do och 2 Ao. Det finns dels som byggsats, med beteckningen K8055 och färdigt kort med beteckningen VM110. Kortet är utmärkt om man vill testa ProviewR med ett enkelt program och få några lampor att blinka. Notera att det inte finns någon watchdog eller stall funktion på kortet.

Kortet är relativt långsamt, det tar ca 25 ms att läsa och skriva till ett kort.

På kortet finns två switchar för adress-inställning, SK5 och SK6. Fyra olika adresser kan ställas in:

Adress	SK5	SK6
0	på	på
1	av	på
2	på	av
3	av	av

Kortet kräver inte någon speciell drivrutin, däremot måste libusb-1.0 installeras. För att kunna läsa och skriva från ProviewR måste oter ha skriv och läsrättigheter på enheten. Kortet kommer att visas under /dev/bus/usb, t ex /dev/usb/002/003. Med kommandot

```
> sudo chmod a+rw /dev/usb/002/003
```

tillåter man alla att skriva och läsa kortet.

Velleman K8055 är testat på Ubuntu 10.4. Det fungerar inte på Debian Lenny.

Kortet konfigureras i ProviewR med agentobjektet USB_Agent, rackobjektet Velleman_K8055 och kortobjektet Velleman_K8055_Board.

Agentobjekt

USB_Agent

USB_Agent läggs under node-objektet och är ett generellt objekt för enheter som använder libusb för åtkomst. Ange Process (Plc) och plc-tråd i PlcThread attributet.

Rackobjekt

Velleman_K8055

Under USB_Agent objektet läggs ett Velleman_K8055 objekt. Även här måste Process och PlcThread anges.

Kortobjekt

Velleman_K8055_Board

Under rackobjektet läggs kortobjekt av typen Velleman_K8055_Board. Man kan ha upp till 4 kort i ett system. Ange Process och PlcThread, och lägg in kortets adress i Address attributet.

Kanaler

Alla kanalobjekt ligger internt i kortobjektet, Velleman_K8055_Board. Där finns en vektor med två

ChanAi objekt, en vektor med 5 ChanDi objekt, en vektor med 2 ChanAo objekt, och en vektor med 8 ChanDo objekt. Koppla kanalobjekten till lämpliga signal objekt.

Ai konfiguration

Ai kanalerna har råvärdesområde 0 – 255 och signalområde 0 – 5 V, dvs RawValRange och ChannelSigValRange ska sättas till

RawValRangeLow	0
RawValRangeHigh	255
ChannelSigValRangeLow	0
ChannelSigValRangeHigh	5

Till exempel för att få ActualValue området 0 – 100 sätt SensorSigValRange 0 - 5 och ActValRange 0 – 100.

Ao konfiguration

Ao kanalerna har råvärdesområde 0 – 255 och signalområde 0 – 5 V, dvs RawValRange och ChannelSigValRange ska sättas till

RawValRangeLow	0
RawValRangeHigh	255
ChannelSigValRangeLow	0
ChannelSigValRangeHigh	5

Till exempel för att få ActualValue området 0 – 100 sätt SensorSigValRange 0 - 5 och ActValRange 0 – 100.

Länkfil

Arkivet libusb-1.0 måste länkas med plcprogrammet. Detta gör man genom att skapa filen \$pwrp_exe/plc_'nodnamn'_'busnr'.opt, t ex \$pwrp_exe/plc_mynode_0517.opt med innehållet

```
$pwr_obj/rt_io_user.o -lpwr_rt -lusb-1.0 -lpwr_usbio_dummy -lpwr_pnak_dummy
```

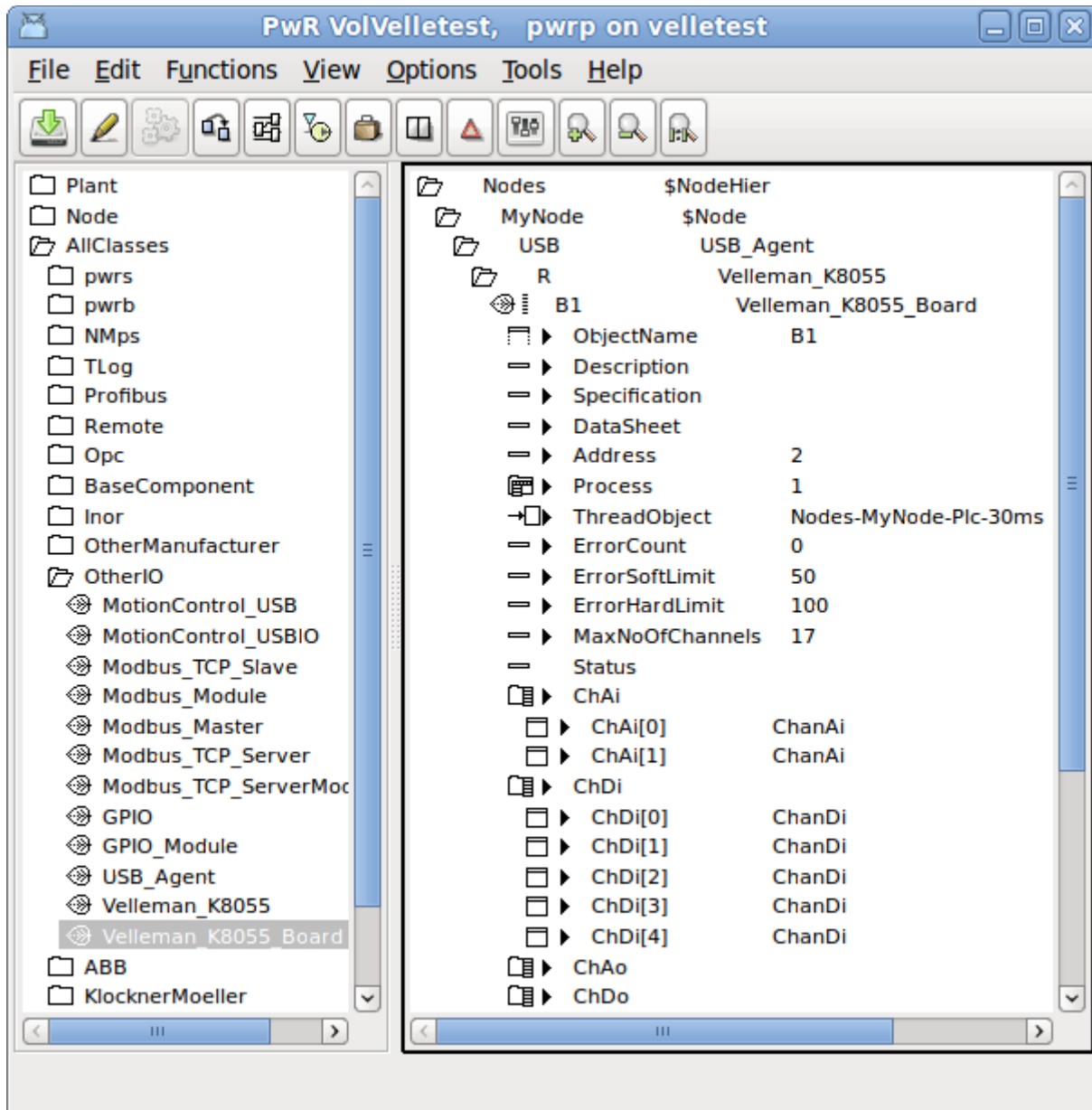


Fig Velleman K8055 konfigurering

Adaption av I/O system

I detta avsnitt beskrivs hur man inför nya I/O system i ProviewR.

Att lägga in ett nytt I/O system kräver kunskap i hur man skapar klasser i ProviewR, samt baskunskap i c programmering.

Ett I/O system kan läggas in för ett enskilt projekt eller ett antal projekt, eller i ProviewR's bassystem. I det första fallet räcker det med att installera ProviewR's utvecklingsmiljö. I det senare fallet måste man installera och bygga från ProviewR's källkod.

Översikt

I/O hanteringen i ProviewR består av ett ramverk som identifierar I/O objekt på en processnod, och anropar I/O objektens metoder för att hämta och ställa ut data.

Nivåer

I/O objekten i en processnod konfigureras i tre nivåer: agent, rack och kort. Ibland finns även en fjärde nivå närvarande: kanal. Kanalobjekten kan konfigureras som individuella objekt, eller existerar som interna objekt i ett kort-objekt.

Till agent-, rack- och kortobjekten kan man registrera metoder. Metoderna kan vara av typen Init, Close, Read, Write och Swap, och anropas av I/O ramverket i en specifik ordning. Funktionaliteten hos ett I/O objekt utgörs av objektets attribut, och de registrerade metoderna för objektet. Allt I/O ramverket gör är att identifiera objekten, välja ut de objekt som är giltiga för den aktuella processen, och anropa metoderna för dessa objekt i en specifik ordning.

Betrakta ett centraliserat I/O system med digitala ingångskort (Di) och digitala utgångskort (Do) monterade på processnodens lokala bus. I det här fallet är agent nivån överflödigt och \$Node objektet rycker in som ställföreträdande agentobjekt. Under \$Node-objektet läggs ett rack-objekt med en open och en close metod. Open metoden knyter upp sig mot drivrutinen för korten. Under rackobjektet konfigureras kortobjekt för Di och Do korten. Di korten har en Open och en Close metod som initierar resp stänger ner kortet, och en Read metod som hämtar värdet på kortets ingångar. Do-kort objekten har också Open och Close metoder, samt en Write metod som ställer ut lämpliga värden på kortens utgångar.

Om vi tittar på ett annat I/O system, Profibus, är nivåerna inte lika lätta att identifiera som i föregående exempel. Profibus är ett distribuerat system, med ett masterkort monterat på den lokala PCI-bussen, som kommunicerar via en seriell förbindelse med slavar placerade ute i anläggningen. Varje slav kan innehålla moduler av olika typ, t ex en modul med 4 Di kanaler, och en med 2 Ao kanaler. I det här fallet representerar masterkortet agentnivån, slaverna racknivån och modulerna kortnivån.

Agent, rack och kort nivåerna är mycket flexibla, och definieras huvudsakligen av attributen och metoderna för I/O systemet klasser. Det gäller inte på kanalnivån, som består av objekt av klasserna ChanDi, ChanDo, ChanAi, ChanAo, ChanLi, ChanIo och ChanCo. Uppgiften för ett kanalobjekt är att representera ett ut eller ingångs värde på I/O enheten och överföra detta värde till det signalobjekt som är kopplat till kanalobjektet. Signalobjekt ligger i anläggningshierarkin och representerar en t ex en givare eller en order till ett ställdon i anläggningen. Liksom det finns en fysisk förbindelse mellan givaren i anläggningen och kanalen på I/O kortet, kopplas även signalobjektet ihop med kanalobjektet. Plcprogram, HMI och applikationer refererar alla signalobjektet som representerar komponenten i anläggningen, inte kanalobjektet som representerar en kanal på en I/O enhet.

Area objekt

Värden som läses in från ingångsenheter och värden som ställs ut till utgångsenheter lagras i speciella areaobjekt. Areaobjekten skapas dynamiskt i runtime och ligger i systemvolymen under hierarkin *pwrNode-active-io*. Det finns ett area objekt för varje signaltyp. Normalt refererar man värdet på en signal genom signalens ActualValue attribut, men detta attribut innehåller i själva verket en pekare som pekar in i areaobjektet. Attributet ValueIndex anger vilket index i in areaobjekt som signalvärdet återfinns på. Orsaken till konstruktionen med areaobjekt är att man under exekveringen av ett logiknät inte vill ha förändringar i signalvärden. Varje plc-tråd tar därför en kopia av areaobjekten innan exekveringen startar och läser signalvärden från kopian, däremot skrivs beräknade signalvärden i areaobjektet.

I/O objekt

Konfigureringen av I/O görs i nodehierarkin under \$Node-objektet. Till varje typ av komponent i I/O hierarkin skapar man en klass som innehåller attribut och metoder. Metoderna är av typen Open, Close, Read, Write och Swap och anropas av I/O-ramverket. Metoderna knyter upp sig mot bussen och läser in data som överförs till areaobjekten, eller hämtar data från areaobjekten som ställs ut på bussen.

Processer

Det finns två systemprocesser i ProviewR som anropar I/O ramverket: plc processen och rt_io_comm. I plc processen gör varje tråd en initiering av I/O ramverket, vilket medför att I/O enheter kan läsas och skrivas synkront med exekveringen av plc-koden för respektive tråd.

Ramverk

I/O-ramverket huvudsakliga uppgift är att identifiera I/O-objekt och anropa de metoder som finns registrerade för objekten.

En första initiering av I/O sker vid uppstart av runtimemiljön, när areaobjekten skapas och varje signal blir tilldelad en plats i areaobjektet. Dessutom kontrolleras kopplingen mellan signal och kanal. Signaler och kanaler har kopplats i utvecklingsmiljön på så sätt att identiteten för kopplad kanal har lagts i signalens SigChanCon attribut. Nu läggs signalens identitet in i kanalen SigChanCon så att man enkelt kan gå från kanal till signal.

Nästa initiering sker av varje process som vill knyta upp sig mot I/O-hantering. Plc-processen och rt_io_comm gör den här initieringen, men det är öppet även för applikationer som vill läsa eller skriva direkt mot I/O enheter att knyta upp sig. Vid initieringen läggs upp en datastruktur med alla agenter, rack, kort och kanaler som ska hanteras av just den här processen, och init-metoderna för dem anropas. Processen anropar sedan cykliskt en read- och en write-funktion, som anropar read-resp write-metoderna för I/O-objekten i datastrukturen.

När man utökar ProviewR med nya I/O-metoder identifierar ramverket objekten och anropar metoderna i rätt ordning, men stallhantering implementeras i metoderna.

Det innebär att Agent/Rack/Kort-metoderna ansvarar för detektering av kommunikationsfel, felräkning och stallaktioner.

När gränsvärden passeras i metodkoden ska flaggor och objektreferenser i IOHandler sättas.

```
ctx->IOHandler->CardErrorSoftLimit = 1;  
ctx->IOHandler->ErrorSoftLimitObject = cdh_ObjIdToAref(...);  
ctx->IOHandler->CardErrorHardLimit = 1;  
ctx->IOHandler->ErrorHardLimitObject = cdh_ObjIdToAref(...);  
För EmergencyBreak sätt nodflaggan i metodkoden.
```

```
ctx->Node->EmergBreakTrue = 1;  
För ResetInputs måste metodkoden uttryckligen nollställa drivrutinens indataavbild. Ramverket känner inte till transportspecifika buffertar och kan inte nollställa dem generellt.
```

Metoder

Metoderna har som uppgift att initiera I/O-systemet, utföra läsning och utställning till I/O-enheterna, och slutligen att koppla ner. Hur de har uppgifterna fördelas beror på I/O systemet uppbyggnad. I ett centraliserat I/O på den lokala bussen, kan metoderna för olika kortobjekt själva gå ut och läsa resp skriva data till sin enhet, och metoderna för agent- och rack-objekten för en ganska lugn tillvaro. I ett distribuerat I/O kommer informationen för enheterna ofta samlade i ett paket, och det blir metoden för agent- eller rack-objektet som tar emot paketet och fördelar innehållet på olika kort-objekt. I kort-objektets metod lägger man lämpligen uppgiften att identifiera data för enskilda kanaler, och utföra eventuell konvertering och läsa resp skriva data i areaobjekten.

Ramverk

En process kan initiera I/O ramverket genom att anropa `io_init()`. Som argument skickar man en bitmask som anger vilken process man är, och trådarna i `plcprocessen` anger även aktuell tråd. `io_init()` utför följande

- skapar en kontext.
- lägger upp en hierarkisk datastruktur av I/O objekt med nivåerna agent, rack, kort och kanal. För agenter allokeras en struct av typen `io_sAgent`, för rack en struct av typ `io_sRack`, för kort en struct av typ `io_sCard`, och slutligen för kanal en struct av typ `io_sChannel`.
- letar upp alla I/O objekt och kontrollerar Process attributet. Om Process attributet matchar den process som skickats med som argument till `io_init()`, läggs objektet in i datastrukturen. Om objektet har ett underliggande I/O objekt som matchar processen läggs det också in i datastrukturen. För `plcprocessen` kontrollerar man dessutom att tråd-argumentet i `io_init()` matchar ThreadObject-attributet i I/O-objektet. Resultatet blir en länkad trädstruktur med de agent, rack, kort och kanal objekt som ska hanteras av den aktuella processen.
- För varje I/O-objekt som läggs in, identifieras metoderna, och pekare till metod-funktionerna hämtas upp. Dessutom hämtas en pekare till objektet upp och objektsnamnet läggs in i datastrukturen.
- `init`-metoden för I/O objekten i datastrukturen anropas. Metoden för första agenten anropas först, därefter agentens första rack, rackets första kort, andra kort osv.

När initieringen är gjord kan processen anropa `io_read()` för att läsa från de I/O enheter som finns med i datastrukturen, och `io_write()` för att ställ ut värden. En tråd i `plcprocessen` anropar `io_read()` varje scan för att hämta in nya värden från processen. Därefter exekveras `plc-koden` och slutligen anropas `io_write()` för att ställa ut nya värden. Read-metoderna anropas i samma ordning som `init`-metoderna, och `write`-metoderna i omvänd ordning.

När processen terminerar, anropar den `io_close()` som i sin tur anropar `close`-metoderna för objekten i datastrukturen. `Close`-metoderna anropas i omvänd ordning jämfört med `init`-metoderna.

Vid en mjuk omstart gör även en omstart av I/O hanteringen. Först anropas `close`-metoderna, därefter anropas `Swap`-metoderna under den tiden omstarten pågår, och därefter `init`-metoderna. Anropet av `swap`-metoderna görs av processen `rt_io_comm`.

`io_init`, funktion för initering av ramverket

```
pwr_tStatus io_init(  
    io_mProcess    process,  
    pwr_tObjid     thread,  
    io_tCtx        *ctx,  
    int            relativ_vector,  
    float          scan_time  
);
```

`io_sCtx`, ramverkets kontext

```
struct io_sCtx {
```

```

io_sAgent      *agentlist;      /* List of agent structures */
io_mProcess    Process;        /* Callers process number */
pwr_tObjid     Thread;         /* Callers thread objid */
int            RelativVector;   /* Used by plc */
pwr_sNode      *Node;          /* Pointer to node object */
pwr_sClass_IOHandler *IOHandler; /* Pointer to IO Handler object */
float          ScanTime;       /* Scantime supplied by caller */
io_tSupCtx     SupCtx;         /* Context for supervise object lists */
};

```

Datastruktur för en agent

```

typedef struct s_Agent {
    pwr_tClassId    Class;        /* Class of agent object */
    pwr_tObjid      Objid;        /* Objid of agent object */
    pwr_tOName      Name;        /* Full name of agent object */
    io_mAction      Action;       /* Type of method defined (Read/Write)*/
    io_mProcess     Process;      /* Process number */
    pwr_tStatus     (* Init) ();  /* Init method */
    pwr_tStatus     (* Close) (); /* Close method */
    pwr_tStatus     (* Read) ();  /* Read method */
    pwr_tStatus     (* Write) (); /* Write method */
    pwr_tStatus     (* Swap) ();  /* Write method */
    void           *op;          /* Pointer to agent object */
    pwr_tDlId       DlId;        /* DlId for agent object pointer */
    int             scan_interval; /* Interval between scans */
    int             scan_interval_cnt; /* Counter to detect next time to scan */
    io_sRack        *racklist;   /* List of rack structures */
    void           *Local;       /* Pointer to method defined data structure*/
    struct s_Agent *next;        /* Next agent */
} io_sAgent;

```

Datastruktur för ett rack

```

typedef struct s_Rack {
    pwr_tClassId    Class;        /* Class of rack object */
    pwr_tObjid      Objid;        /* Objid of rack object */
    pwr_tOName      Name;        /* Full name of rack object */
    io_mAction      Action;       /* Type of method defined (Read/Write)*/
    io_mProcess     Process;      /* Process number */
    pwr_tStatus     (* Init) ();  /* Init method */
    pwr_tStatus     (* Close) (); /* Close method */
    pwr_tStatus     (* Read) ();  /* Read method */
    pwr_tStatus     (* Write) (); /* Write method */
    pwr_tStatus     (* Swap) ();  /* Swap method */
    void           *op;          /* Pointer to rack object */
    pwr_tDlId       DlId;        /* DlId för rack object pointer */
    pwr_tUInt32     size;        /* Size of rack data area in byte */
    pwr_tUInt32     offset;      /* Offset to rack data area in agent */
    int             scan_interval; /* Interval between scans */
    int             scan_interval_cnt; /* Counter to detect next time to scan */
    int             AgentControlled; /* TRUE if kontrollert by agent */
    io_sCard        *cardlist;   /* List of card structures */
    void           *Local;       /* Pointer to method defined data structure*/
    struct s_Rack  *next;        /* Next rack */
} io_sRack;

```

Datastruktur för ett kort

```

typedef struct s_Card {
    pwr_tClassId    Class;        /* Class of card object */
    pwr_tObjid      Objid;        /* Objid of card object */
    pwr_tOName      Name;        /* Full name of card object */

```

```

io_mAction      Action;          /* Type of method defined (Read/Write)*/
io_mProcess     Process;         /* Process number */
pwr_tStatus     (* Init) ();      /* Init method */
pwr_tStatus     (* Close) ();     /* Close method */
pwr_tStatus     (* Read) ();      /* Read method */
pwr_tStatus     (* Write) ();     /* Write method */
pwr_tStatus     (* Swap) ();      /* Write method */
pwr_tAddress     *op;             /* Pointer to card object */
pwr_tDlId       DlId;            /* DlId for card object pointer */
pwr_tUInt32     size;            /* Size of card data area in byte */
pwr_tUInt32     offset;          /* Offset to card data area in rack */
int             scan_interval;   /* Interval between scans */
int             scan_interval_cnt; /* Counter to detect next time to scan */
int             AgentControlled; /* TRUE if kontrollered by agent */
int             ChanListSize;    /* Size of chanlist */
io_sChannel     *chanlist;       /* Array of channel structures */
void            *Local;          /* Pointer to method defined data structure*/
struct s_Card   *next;           /* Next card */
} io_sCard;

```

Datastruktur för en kanal

```

typedef struct {
    void            *cop;          /* Pointer to channel object */
    pwr_tDlId       ChanDlId;      /* DlId for pointer to channel */
    pwr_sAttrRef    ChanAref;      /* AttrRef for channel */
    void            *sop;          /* Pointer to signal object */
    pwr_tDlId       SigDlId;       /* DlId for pointer to signal */
    pwr_sAttrRef    SigAref;       /* AttrRef for signal */
    void            *vbp;          /* Pointer to valuebase for signal */
    void            *abs_vbp;      /* Pointer to absvaluebase (Co only) */
    pwr_tClassId    ChanClass;     /* Class of channel object */
    pwr_tClassId    SigClass;      /* Class of signal object */
    pwr_tUInt32     size;          /* Size of channel in byte */
    pwr_tUInt32     offset;        /* Offset to channel in card */
    pwr_tUInt32     mask;          /* Mask for bit oriented channels */
} io_sChannel;

```

Skapa I/O-objekt

I en processnod konfigureras I/O-systemet i nodhierarkin med objekt av typen agent, rack och kort. Klasserna för de här objekten skapar man i klasseditorn. Klasserna definieras med ett \$ClassDef objekt, ett \$ObjBodyDef objekt (RtBody), och under detta med ett \$Attribute objekt för varje attribut i klassen. Attributen bestäms av funktionaliteten i metoderna för klassen, men det finns några generella attribut (Process, ThreadObject och Description). I \$ClassDef-objektets Flag-ord ska anges om det är ett agent, rack eller kort-objekt, och metoderna definieras med speciella *Method* objekt.

Det är ganska vanligt att flera klasser i ett I/O-system delar attribut och kanske även metoder. Ett ingångskort som finns med olika antal ingångar, kan ofta använda samma metod. Det som skiljer är antalet kanalobjekt. De övriga attributen kan då läggas i en basklass, som även innehåller metodobjekten. Subklasser ärver både attributen och metoderna, det som tillkommer är kanalobjekten, som kan läggas som enskilda attribut, eller, om de är av samma typ, som en vektor av kanalobjekt. Om kanalerna läggs som vektor eller som enskilda attribut påverkas av hur man vill att referensen i plcdokumenterna ska se ut. Med en array får man indexering från 0, med enskilda attribut kan man styra namngivningen själv.

I exemplet nedan visas en basklass i Fig *Exempel på en basklass för ett kort* och en subklass i Fig *Exempel på en kortklass med en superklass och 32 kanalobjekt*. Basklassen Ssab_BaseDiCard innehåller alla attribut som används av metoderna och I/O-ramverket. Subklassen Ssab_DI32D innehåller super-attributet med TypeRef Ssab_BaseDiCard, och 32 kanalattribut av typen ChanDi. Eftersom indexeringen av den här korttypen av tradition går från 1 har man valt att lägga kanalerna

som enskilda attribut, men de kan också läggas som en vektor av typen ChanDi.

Fig Exempel på en basklass för ett kort

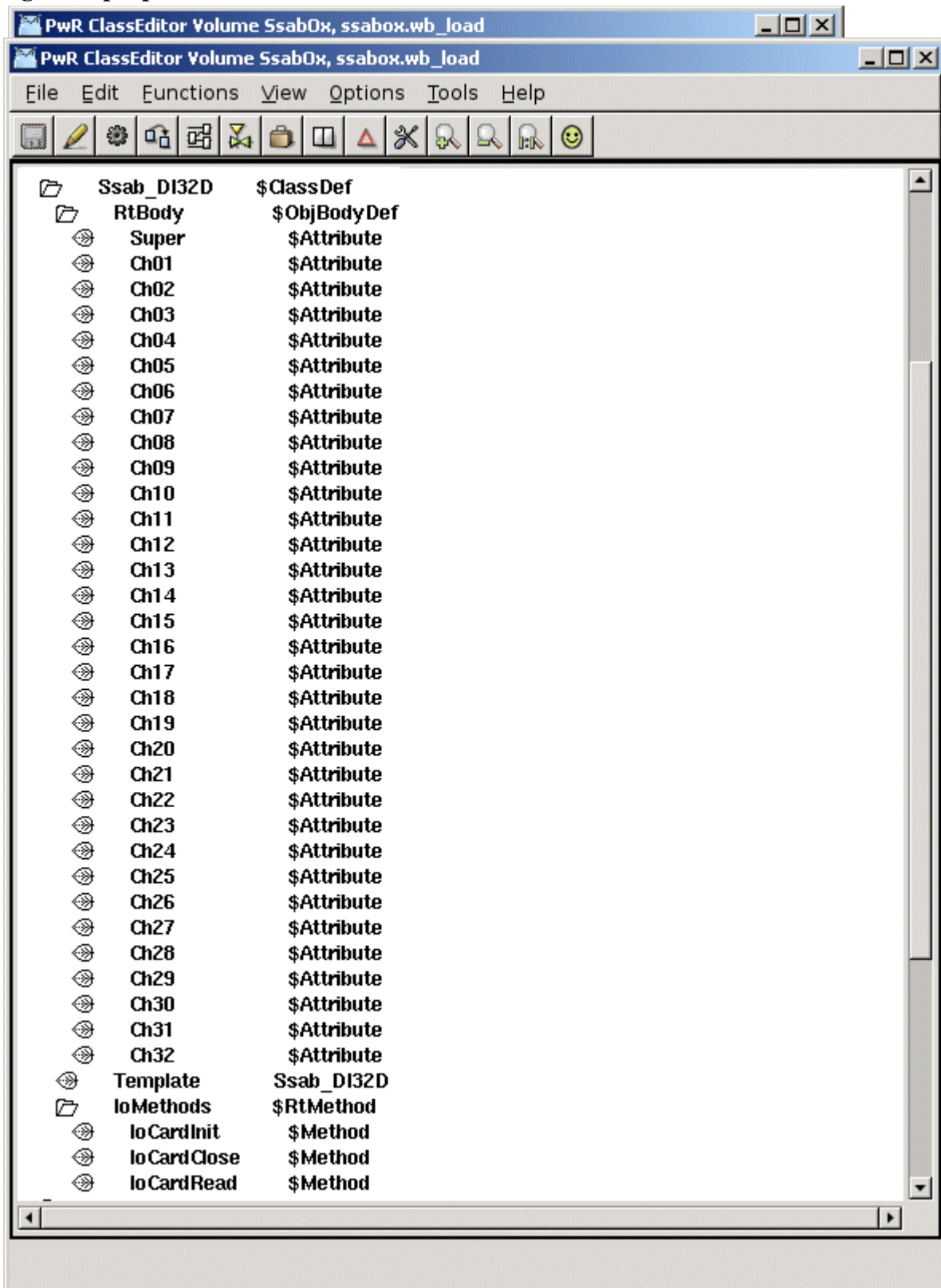


Fig Exempel på en kortklass med superklass och 32 kanalobjekt

Flags

I \$ClassDef objektets Flags attribut ska IOAgent biten sättas för agent-klasser, IORack biten för rack-klasser och IOCard biten för kort-klasser.

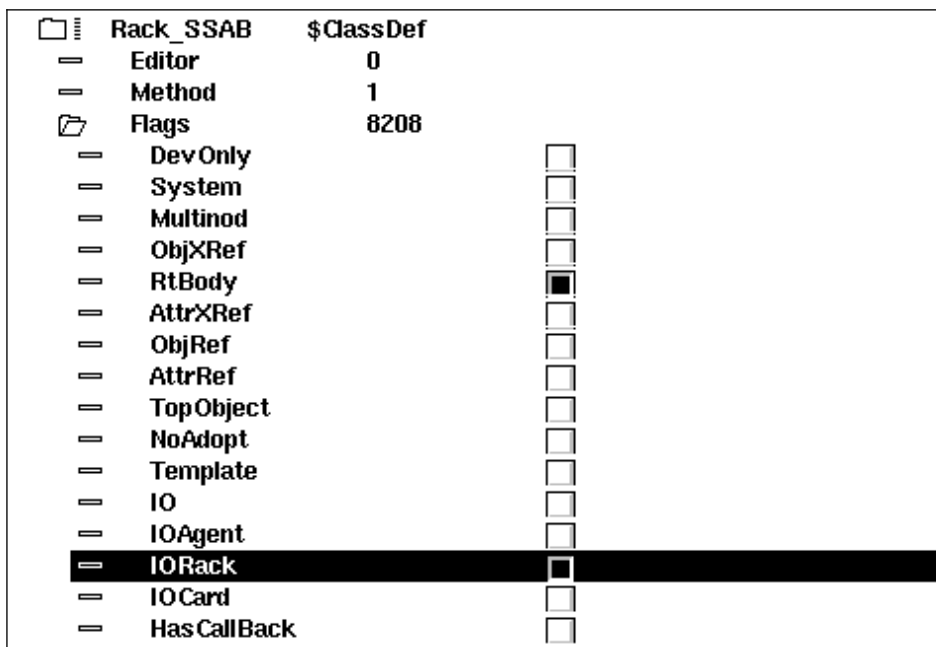


Fig IORack biten satt för en rack klass

Attribut

Description

Attribut av typ pwr:Type-\$String80. Innehållet visas som beskrivning i navigatören.

Process

Attribut av typ pwr:Type-\$UInt32. Anger vilken process som ska hantera enheten.

ThreadObject

Attribut av typ pwr:Type-\$Objid. Anger vilken tråd i plcprocessen som ska hantera enheten.

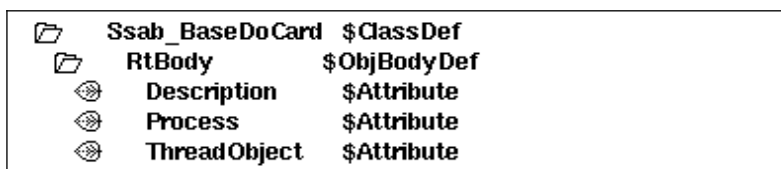


Fig Standard attribut

Metod objekt

Metodobjekten används för att identifiera metoderna för klassen. Metoderna utgörs av c-funktioner som registreras i c-koden med ett namn, en sträng som består av klassnamn och metodnamn, t ex "Ssab-AIuP-IOCardInit". Namnet läggs även in i ett metodobjekt i klassbeskrivning och gör att I/O-ramverket kan hitta rätt c-funktion för klassen.

Under \$ClassDef objektet läggs ett \$RtMethod objekt men namnet IoMethods. Under detta läggs ett \$Method objekt för varje metod som ska definieras för klassen. I attributet MethodName anges namnet för metoden.

Agenter

För agenter skapas \$Method objekt med namnen IoAgentInit, IoAgentClose, IoAgentRead och IoAgentWrite.

Rack

För rack skapas \$Method objekt med namn IoRackInit, IoRackClose, IoRackRead och IoRackWrite.

Kort

För kort skapas \$Method objekt med namn IoCardInit, IoCardClose, IoCardRead och IoCardWrite.

	Ssab_AI32uP	\$ClassDef
	RtBody	\$ObjBodyDef
	Template	Ssab_AI32uP
	IoMethods	\$RtMethod
	IoCardInit	\$Method
	MethodName	Ssab_AiuP - IoCardInit
	MethodArguments	
	IoCardClose	\$Method
	MethodName	Ssab_AiuP - IoCardClose
	MethodArguments	
	IoCardRead	\$Method
	MethodName	Ssab_AiuP - IoCardRead
	MethodArguments	

Fig Metodobjekt

Connect-metod för ThreadObject

När trådobjektet i attributet ThreadObject ska anges för en instans, kan det matas inför hand, men man kan även definiera en meny-metod som lägger in ett utvalt trådobjekt i attributet. Metoden aktiveras från popupmenyn för IO-objektet i konfiguratören.

Metoden definieras i klassbeskrivningen med \$Menu och \$MenuButton objekt, se *Fig Connect metod*. Under \$ClassDef objektet läggs ett \$Menu objekt med namnet ConfiguratorPoson. Under detta ytterligare ett \$Menu objekt med namnet Pointed, och under detta ett \$MenuButton objekt med namnet Connect. Ange ButtonName (texten i popupmenyn för metoden) och MethodName och FilterName. Metoden och filtret som används finns definierade i \$Objid klassen. MethodName ska vara \$Objid-Connect och FilterName \$Objid-IsOkConnected.

Fig Connect metod

	Ssab_BaseDoCard	\$ClassDef
	RtBody	\$ObjBodyDef
	Template	Ssab_BaseDoCard
	ConfiguratorPoson	\$Menu
	Pointed	\$Menu
	Connect	\$MenuButton
	ButtonName	Connect PlcThread
	MethodName	\$Objid - Connect
	MethodArguments	
	FilterName	\$Objid - IsOkConnect
	FilterArguments	

Metoder

För agent, rack och kort klasserna skriver man metoder i programmeringsspråket c. En metod är en c-funktion som är gemensam för en klass (eller flera klasser) och som anropas av I/O-ramverket för alla instanser av en klassen. För att I/O-hanteringen ska bli så flexibel som möjlig, utför metoderna det mesta av I/O-hanterings jobbet. Ramverkets uppgift är egenligen bara att identifiera de olika

I/O-objekten och anropa metoderna för dessa, samt tillhandahålla lämpliga datastrukturer för metoderna.

Det finns fem typer av metoder: Init, Close, Read, Write och Swap.

- Init-metoden anropas vid initieringen av I/O-hanteringen, dvs vid uppstart av runtime miljön och vid en mjuk omstart.
- Close-metoden anropas när I/O-hanteringen avslutas, dvs när runtime-miljön stoppas och vid en mjuk omstart.
- Read-metoden anropas cykliskt när det är dags att läsa av ingångkort.
- Write-metoden anropas cykliskt när det är dags att ställa ut värden till utgångskorten.
- Swap-metoden anropas under en mjuk omstart.

Lokal datastruktur

I datastrukturerna io_sAgent, io_sRack och io_sCard finns ett element Local där metoden kan lagra en pekare till lokal data för en I/O-enhet. Lokala data allokeras i init-metoden och finns sedan tillgängligt vid varje metoanrop.

Agent-Metoder

IoAgentInit

Initierings metod för en agent.

```
static pwr_tStatus IoAgentInit( io_tCtx      ctx,  
                               io_sAgent    *ap)
```

IoAgentClose

Close metod för en agent.

```
static pwr_tStatus IoAgentClose( io_tCtx      ctx,  
                                io_sAgent    *ap)
```

IoAgentRead

Read metod för en agent.

```
static pwr_tStatus IoAgentRead( io_tCtx      ctx,  
                                io_sAgent    *ap)
```

IoAgentWrite

Write metod för en agent.

```
static pwr_tStatus IoAgentWrite( io_tCtx      ctx,  
                                io_sAgent    *ap)
```

IoAgentSwap

Swap metod för en agent.

```
static pwr_tStatus IoAgentSwap( io_tCtx      ctx,  
                                io_sAgent    *ap)
```

Rack-metoder

IoRackInit

```
static pwr_tStatus IoRackInit( io_tCtx      ctx,
```

```
io_sAgent    *ap,  
io_sRack     *rp)
```

IoRackClose

```
static pwr_tStatus IoRackClose( io_tCtx    ctx,  
                                io_sAgent  *ap,  
                                io_sRack    *rp)
```

IoRackRead

```
static pwr_tStatus IoRackRead( io_tCtx    ctx,  
                               io_sAgent  *ap,  
                               io_sRack    *rp)
```

IoRackWrite

```
static pwr_tStatus IoRackWrite( io_tCtx    ctx,  
                                io_sAgent  *ap,  
                                io_sRack    *rp)
```

IoRackSwap

```
static pwr_tStatus IoRackSwap( io_tCtx    ctx,  
                               io_sAgent  *ap,  
                               io_sRack    *rp)
```

Card-metoder

IoCardInit

```
static pwr_tStatus IoCardInit( io_tCtx    ctx,  
                               io_sAgent  *ap,  
                               io_sRack    *rp,  
                               io_sCard    *cp)
```

IoCardClose

```
static pwr_tStatus IoCardClose( io_tCtx    ctx,  
                                io_sAgent  *ap,  
                                io_sRack    *rp,  
                                io_sCard    *cp)
```

IoCardRead

```
static pwr_tStatus IoCardRead( io_tCtx    ctx,  
                               io_sAgent  *ap,  
                               io_sRack    *rp,  
                               io_sCard    *cp)
```

IoCardWrite

```
static pwr_tStatus IoCardWrite( io_tCtx    ctx,  
                                io_sAgent  *ap,  
                                io_sRack    *rp,  
                                io_sCard    *cp)
```

IoCardSwap

```
static pwr_tStatus IoCardSwap( io_tCtx    ctx,  
                               io_sAgent  *ap,  
                               io_sRack    *rp,  
                               io_sCard    *cp)
```

Registrering av metoder

Metoderna för en klass måste registreras, så att man från metod-objektet i klassbeskrivningen kan hitta rätt funktioner för en specifik klass. Nedan visas ett exempel på hur metoderna IoCardInit, IoCardClose och IoCardRead registreras för klassen Ssab_AiuP.

```
pwr_dExport pwr_BindIoMethods(Ssab_AiuP) = {
    pwr_BindIoMethod(IoCardInit),
    pwr_BindIoMethod(IoCardClose),
    pwr_BindIoMethod(IoCardRead),
    pwr_NullMethod
};
```

Registrering av klassen

Dessutom måste klassen registreras. Det här sker på olika sätt beroende på om I/O systemet är implementerat som en modul i ProviewR's bassystem, eller som en del i ett projekt.

Modul i ProviewR's bassystem

Är I/O systemet implementerat som en modul i ProviewR's bassystem, skapar man en fil, lib/rt/src/rt_io_'modulnamn'.meth, och listar alla klasser som har registrerade metoder i denna.

Projekt

Om I/O systemet är en del av en projekt, sker registreringen i en c-modul som länkas med plc-programmet. I exemplet nedan registreras klasserna Ssab_Rack och Ssab_Aiup i filen rt_io_user.c.

```
#include "pwr.h"
#include "rt_io_base.h"

pwr_dImport pwr_BindIoUserMethods(Ssab_Rack);
pwr_dImport pwr_BindIoUserMethods(Ssab_Aiup);

pwr_BindIoUserClasses(User) = {
    pwr_BindIoUserClass(Ssab_Rack),
    pwr_BindIoUserClass(Ssab_Aiup),
    pwr_NullClass
};
```

Filen kompileras och länkas med plc-programmet genom att en länk-fil skapas på \$pwrp_exe. Filen ska namnges plc_'nodnamn'_'busnr'.opt, tex plc_mynode_0517.opt. Innehållet i filen skickas med som indata till länkaren, ld, och man måste även ta med modulerna med metoderna för klassen. I exemplet nedan antas att dessa moduler ligger i arkivet \$pwrp_lib/libpwrp.a.

```
$pwr_obj/rt_io_user.o -lpwrp -lpwr_usbio_dummy -lpwr_usb_dummy -lpwr_pnak_dummy
```

Exempel på rack metoder

```
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>

#include "pwr.h"
#include "pwr_baseclasses.h"
#include "pwr_ssaboxclasses.h"
#include "rt_io_base.h"
#include "rt_errh.h"
#include "rt_io_rack_init.h"
#include "rt_io_m_ssab_locals.h"
#include "rt_io_msg.h"
```

```

/* Init method */
static pwr_tStatus IoRackInit( io_tCtx ctx,
                              io_sAgent *ap,
                              io_sRack *rp)
{
    io_sRackLocal *local;

    /* Open Qbus driver */
    local = calloc( 1, sizeof(*local));
    rp->Local = local;

    local->Qbus_fp = open("/dev/qbus", O_RDWR);
    if ( local->Qbus_fp == -1) {
        errh_Error( "Qbus initialization error, IO rack %s", rp->Name);
        ctx->Node->EmergBreakTrue = 1;
        return IO__ERRDEVICE;
    }

    errh_Info( "Init of IO rack %s", rp->Name);
    return 1;
}

/* Close method */
static pwr_tStatus IoRackClose( io_tCtx ctx,
                              io_sAgent *ap,
                              io_sRack *rp)
{
    io_sRackLocal *local;

    /* Close Qbus driver */
    local = rp->Local;

    close( local->Qbus_fp);
    free( (char *)local);

    return 1;
}

/* Every method to be exported to the workbench should be registered here. */
pwr_dExport pwr_BindIoMethods(Rack_SSAB) = {
    pwr_BindIoMethod(IoRackInit),
    pwr_BindIoMethod(IoRackClose),
    pwr_NullMethod
};

```

Exempel på metoder digitalt ingångskort

```

#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>

#include "pwr.h"
#include "rt_errh.h"
#include "pwr_baseclasses.h"
#include "pwr_ssaboxclasses.h"
#include "rt_io_base.h"

```

```

#include "rt_io_msg.h"
#include "rt_io_filter_di.h"
#include "rt_io_ssab.h"
#include "rt_io_card_init.h"
#include "rt_io_card_close.h"
#include "rt_io_card_read.h"
#include "qbus_io.h"
#include "rt_io_m_ssab_locals.h"

/* Local data */
typedef struct {
    unsigned int    Address[2];
    int             Qbus_fp;
    struct {
        pwr_sClass_Di *sop[16];
        void          *Data[16];
        pwr_tBoolean Found;
    } Filter[2];
    pwr_tTime       ErrTime;
} io_sLocal;

/* Init method */
static pwr_tStatus IoCardInit( io_tCtx  ctx,
                              io_sAgent *ap,
                              io_sRack  *rp,
                              io_sCard  *cp)
{
    pwr_sClass_Ssab_BaseDiCard *op;
    io_sLocal                  *local;
    int                        i, j;

    op = (pwr_sClass_Ssab_BaseDiCard *) cp->op;
    local = calloc( 1, sizeof(*local));
    cp->Local = local;

    errh_Info( "Init of di card '%s'", cp->Name);

    local->Address[0] = op->RegAddress;
    local->Address[1] = op->RegAddress + 2;
    local->Qbus_fp = ((io_sRackLocal *) (rp->Local))->Qbus_fp;

    /* Init filter */
    for ( i = 0; i < 2; i++) {
        /* The filter handles one 16-bit word */
        for ( j = 0; j < 16; j++)
            local->Filter[i].sop[j] = cp->chanlist[i*16+j].sop;
        io_InitDiFilter( local->Filter[i].sop, &local->Filter[i].Found,
                        local->Filter[i].Data, ctx->ScanTime);
    }

    return 1;
}

/* Close method */
static pwr_tStatus IoCardClose( io_tCtx  ctx,
                              io_sAgent *ap,
                              io_sRack  *rp,
                              io_sCard  *cp)
{
    io_sLocal                  *local;
    int                        i;

```

```

local = (io_sLocal *) cp->Local;

errh_Info( "IO closing di card '%s'", cp->Name);

/* Free filter data */
for ( i = 0; i < 2; i++) {
    if ( local->Filter[i].Found)
        io_CloseDiFilter( local->Filter[i].Data);
}
free( (char *) local);

return 1;
}

/* Read method */
static pwr_tStatus IoCardRead( io_tCtX   ctx,
                              io_sAgent *ap,
                              io_sRack  *rp,
                              io_sCard  *cp)
{
    io_sLocal          *local;
    io_sRackLocal      *r_local = (io_sRackLocal *) (rp->Local);
    pwr_tUInt16        data = 0;
    pwr_sClass_Ssab_BaseDiCard *op;
    pwr_tUInt16        invmask;
    pwr_tUInt16        convmask;
    int                i;
    int                sts;
    qbus_io_read       rb;
    pwr_tTime          now;

    local = (io_sLocal *) cp->Local;
    op = (pwr_sClass_Ssab_BaseDiCard *) cp->op;

    for ( i = 0; i < 2; i++) {
        if ( i == 0) {
            convmask = op->ConvMask1;
            invmask = op->InvMask1;
        }
        else {
            convmask = op->ConvMask2;
            invmask = op->InvMask2;
            if ( !convmask)
                break;
            if ( op->MaxNoOfChannels == 16)
                break;
        }
    }

    /* Read from local Q-bus */
    rb.Address = local->Address[i];
    sts = read( local->Qbus_fp, &rb, sizeof(rb));
    data = (unsigned short) rb.Data;

    if ( sts == -1) {
        /* Increase error count and check error limits */
        clock_gettime(CLOCK_REALTIME, &now);

        if (op->ErrorCount > op->ErrorSoftLimit) {
            /* Ignore if some time has expired */
            if (now.tv_sec - local->ErrTime.tv_sec < 600)
                op->ErrorCount++;
        }
    }
}

```

```

        else
            op->ErrorCount++;
            local->ErrTime = now;

            if ( op->ErrorCount == op->ErrorSoftLimit)
                errh_Error( "IO Error soft limit reached on card '%s'", cp->Name);
            if ( op->ErrorCount >= op->ErrorHardLimit)
            {
                errh_Error( "IO Error hard limit reached on card '%s', IO stopped", cp-
>Name);
                ctx->Node->EmergBreakTrue = 1;
                return IO__ERRDEVICE;
            }
            continue;
        }

        /* Invert */
        data = data ^ invmask;

        /* Filter */
        if ( local->Filter[i].Found)
            io_DiFilter( local->Filter[i].sop, &data, local->Filter[i].Data);

        /* Move data to valuebase */
        io_DiUnpackWord( cp, data, convmask, i);
    }
    return 1;
}

/* Every method to be exported to the workbench should be registered here. */

pwr_dExport pwr_BindIoMethods(Ssab_Di) = {
    pwr_BindIoMethod(IoCardInit),
    pwr_BindIoMethod(IoCardClose),
    pwr_BindIoMethod(IoCardRead),
    pwr_NullMethod
};

```

Exempel på metoder för digitalt utgångskort

```

#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>

#include "pwr.h"
#include "rt_errh.h"
#include "pwr_baseclasses.h"
#include "pwr_ssaboxclasses.h"
#include "rt_io_base.h"
#include "rt_io_msg.h"
#include "rt_io_filter_po.h"
#include "rt_io_ssab.h"
#include "rt_io_card_init.h"
#include "rt_io_card_close.h"
#include "rt_io_card_write.h"
#include "qbus_io.h"
#include "rt_io_m_ssab_locals.h"

/* Local data */

```



```

typedef struct {
    unsigned int    Address[2];
    int             Qbus_fp;
    struct {
        pwr_sClass_Po *sop[16];
        void          *Data[16];
        pwr_tBoolean Found;
    } Filter[2];
    pwr_tTime       ErrTime;
} io_sLocal;

/* Init method */
static pwr_tStatus IoCardInit( io_tCtx  ctx,
                              io_sAgent *ap,
                              io_sRack  *rp,
                              io_sCard  *cp)
{
    pwr_sClass_Ssab_BaseDoCard *op;
    io_sLocal                   *local;
    int                         i, j;

    op = (pwr_sClass_Ssab_BaseDoCard *) cp->op;
    local = calloc( 1, sizeof(*local));
    cp->Local = local;

    errh_Info( "Init of do card '%s'", cp->Name);

    local->Address[0] = op->RegAddress;
    local->Address[1] = op->RegAddress + 2;
    local->Qbus_fp = ((io_sRackLocal *) (rp->Local))->Qbus_fp;

    /* Init filter for Po signals */
    for ( i = 0; i < 2; i++) {
        /* The filter handles one 16-bit word */
        for ( j = 0; j < 16; j++) {
            if ( cp->chanlist[i*16+j].SigClass == pwr_cClass_Po)
                local->Filter[i].sop[j] = cp->chanlist[i*16+j].sop;
        }
        io_InitPoFilter( local->Filter[i].sop, &local->Filter[i].Found,
                        local->Filter[i].Data, ctx->ScanTime);
    }

    return 1;
}

/* Close method */
static pwr_tStatus IoCardClose( io_tCtx  ctx,
                              io_sAgent *ap,
                              io_sRack  *rp,
                              io_sCard  *cp)
{
    io_sLocal                   *local;
    int                         i;

    local = (io_sLocal *) cp->Local;

    errh_Info( "IO closing do card '%s'", cp->Name);

    /* Free filter data */
    for ( i = 0; i < 2; i++) {
        if ( local->Filter[i].Found)
            io_ClosePoFilter( local->Filter[i].Data);
    }
}

```

```

    }
    free( (char *) local);

    return 1;
}

/* Write method */
static pwr_tStatus IoCardWrite( io_tCtx ctx,
                                io_sAgent *ap,
                                io_sRack *rp,
                                io_sCard *cp)
{
    io_sLocal          *local;
    io_sRackLocal      *r_local = (io_sRackLocal *) (rp->Local);
    pwr_tUInt16        data = 0;
    pwr_sClass_Ssab_BaseDoCard *op;
    pwr_tUInt16        invmask;
    pwr_tUInt16        testmask;
    pwr_tUInt16        testvalue;
    int                i;
    qbus_io_write      wb;
    int                sts;
    pwr_tTime          now;

    local = (io_sLocal *) cp->Local;
    op = (pwr_sClass_Ssab_BaseDoCard *) cp->op;

    for ( i = 0; i < 2; i++) {
        if ( ctx->Node->EmergBreakTrue && ctx->Node->EmergBreakSelect == FIXOUT) {
            if ( i == 0)
                data = op->FixedOutValue1;
            else
                data = op->FixedOutValue2;
        }
        else
            io_DoPackWord( cp, &data, i);

        if ( i == 0) {
            testmask = op->TestMask1;
            invmask = op->InvMask1;
        }
        else {
            testmask = op->TestMask2;
            invmask = op->InvMask2;
            if ( op->MaxNoOfChannels == 16)
                break;
        }

        /* Invert */
        data = data ^ invmask;

        /* Filter Po signals */
        if ( local->Filter[i].Found)
            io_PoFilter( local->Filter[i].sop, &data, local->Filter[i].Data);

        /* Testvalues */
        if ( testmask) {
            if ( i == 0)
                testvalue = op->TestValue1;
            else
                testvalue = op->TestValue2;
            data = (data & ~ testmask) | (testmask & testvalue);
        }
    }
}

```

```

}

/* Write to local Q-bus */
wb.Data = data;
wb.Address = local->Address[i];
sts = write( local->Qbus_fp, &wb, sizeof(wb));

if ( sts == -1) {
    /* Increase error count and check error limits */
    clock_gettime(CLOCK_REALTIME, &now);

    if (op->ErrorCount > op->ErrorSoftLimit) {
        /* Ignore if some time has expired */
        if (now.tv_sec - local->ErrTime.tv_sec < 600)
            op->ErrorCount++;
    }
    else
        op->ErrorCount++;
    local->ErrTime = now;

    if ( op->ErrorCount == op->ErrorSoftLimit)
        errh_Error( "IO Error soft limit reached on card '%s'", cp->Name);
    if ( op->ErrorCount >= op->ErrorHardLimit)
    {
        errh_Error( "IO Error hard limit reached on card '%s', IO stopped", cp-
>Name);
        ctx->Node->EmergBreakTrue = 1;
        return IO__ERRDEVICE;
    }
    continue;
}
}
return 1;
}

/* Every method to be exported to the workbench should be registered here. */
pwr_dExport pwr_BindIoMethods(Ssab_Do) = {
    pwr_BindIoMethod(IoCardInit),
    pwr_BindIoMethod(IoCardClose),
    pwr_BindIoMethod(IoCardWrite),
    pwr_NullMethod
};

```

Steg för steg beskrivning

Detta avsnitt innehåller ett exempel på ett hur ett I/O system läggs in i ProviewR.

I/O systemet är USB I/O som tillverkas av Motion Control. Det består av ett kort med 21 kanaler av olika typ. De fyra första kanalerna är Digitala utgångar av av relätyp som tål upp till 230 V. Nästa fyra kanaler är Digitala utgångar med optokopplare. Nästa åtta kanaler kan antingen konfigureras som digitala utgångar, digitala ingångar eller analoga ingångar. Därefter följer fem kanaler som kan vara digitala ingångar eller utgångar, där den tredje även kan konfigureras som en räknare, och den fjärde och femte som analoga utsignaler. För att inte koden ska bli för komplex, begränsar vi oss till konfigurationen: kanal 0-3 Do, 4-7 Di, 8-15 Ai, 16-18 Di och 19-20 Ao.

Lägga in i ett projekt

I första exemplet lägger vi in I/O systemet i ett projekt. Vi kommer att skapa en klassvolym, och lägga in rack och kort-klasser i denna. Vidare kommer vi att skriva I/O metoder för klasserna och se till att dessa länkas med plcprogrammet. Vi lägger in I/O objekt i nodhierarkin i rotvolymen, installerar drivrutinen för USB I/O, och startar upp I/O hanteringen på processnoden.

Skapa klasser

Skapa en klassvolym

Det första steget är att skapa klasser för I/O objekten. Klasser definieras i klassvolymen, och först måste vi skapa en klassvolym i projektet. Klassvolymen ska först registreras i GlobalVolumeList. Vi startar Administratören med

> pwra

och öppnar GlobalVolumeList genom att aktivera *File/Open/GlobalVolumeList* i menyn. Vi går in i editerings-mod, och skapar ett VolumeReg objekt med namnet *CVolMerk1*. Volymidentiteten för användarklassvolymen ska ligga i intervallet 0.0.2-249.1-254 och vi väljer 0.0.99.20 som identitet för vår klassvolym. I attributet Projekt anges vårt projekt, *mars2*.

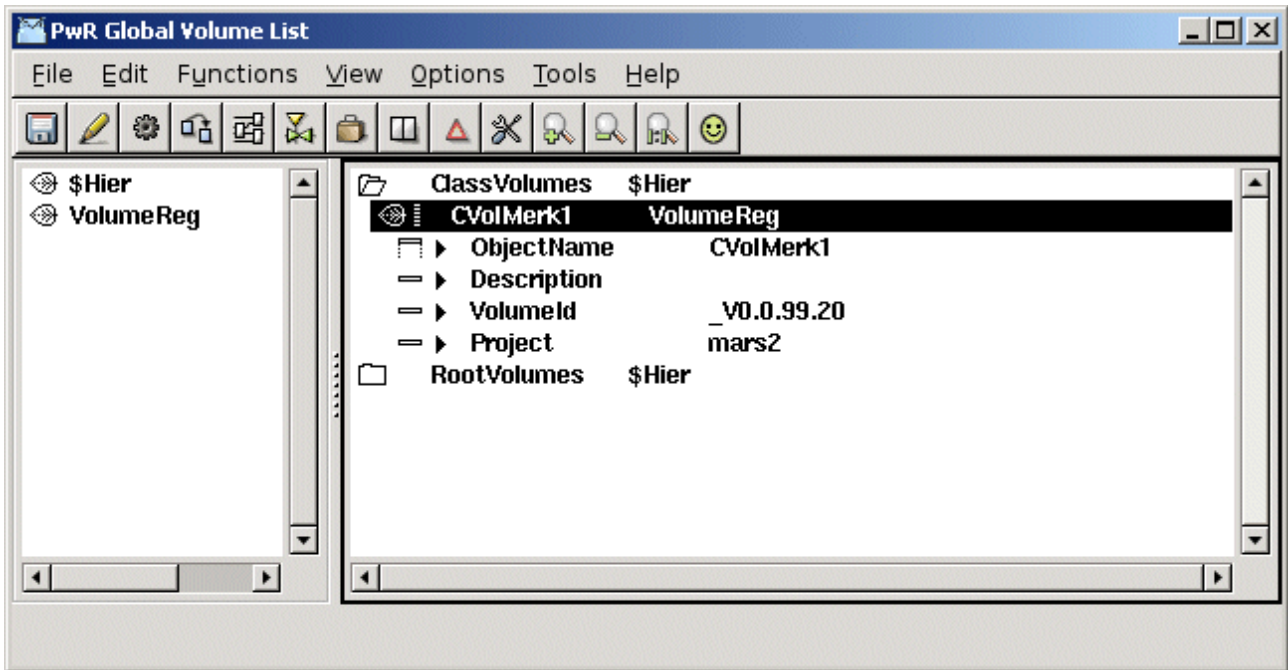


Fig Registrering av klassvolym

Öppna klassvolymen

Vi ska nu konfigurera och skapa klassvolymen i projektet. Det görs i directory-volymen.

Vi går in i directoryvolymen med

> pwrs

går in i editerings-mod och skapar ett objekt av typen ClassVolumeConfig i volymshierakin. Objektet namnges med volymnamnet *CVolMerk1*. När vi har lämnat editerings-mod kan vi öppna klassvolymen genom att aktivera *OpenClassEditor...* i popupmenyn för objektet.

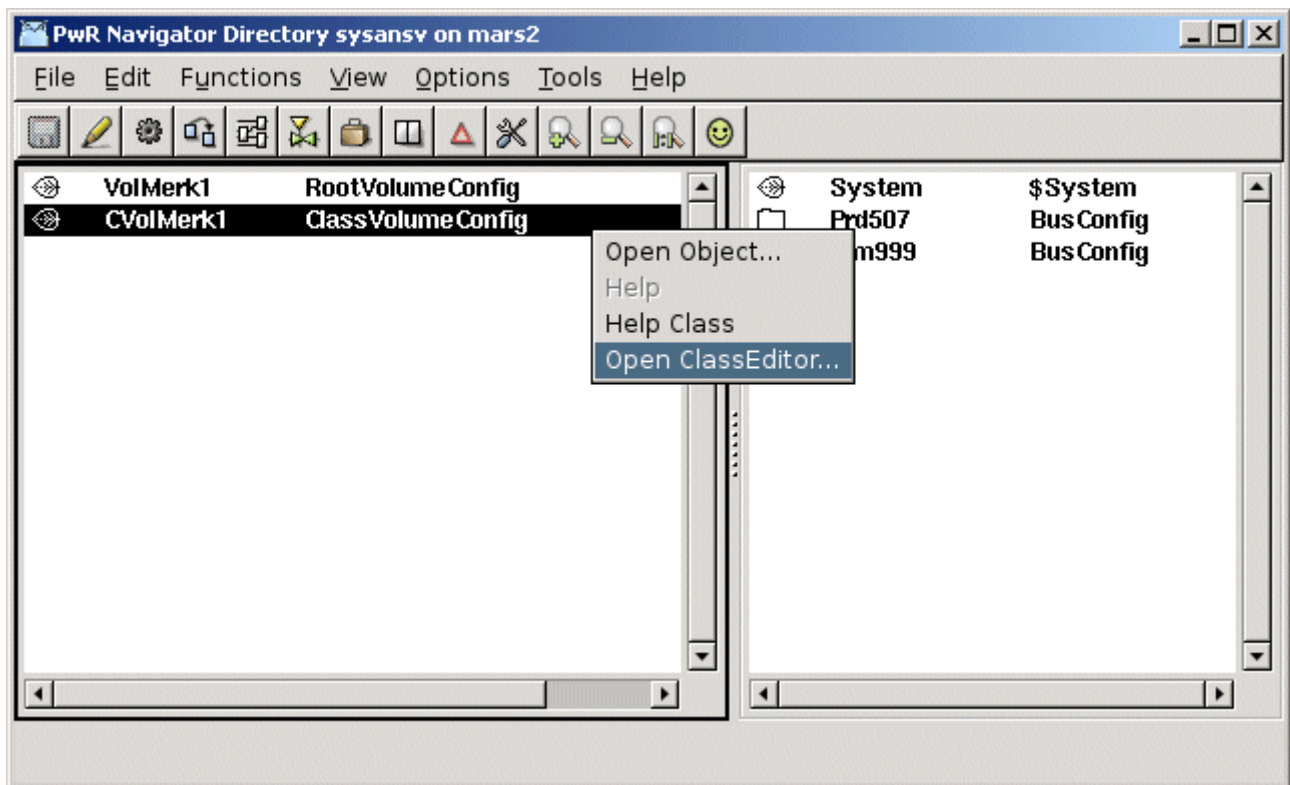


Fig Konfigurering av klassvolym i Directory volymen, och start av klasseditorn

I klasseditor definieras klasser med hjälp av speciella klassdefinitions objekt. Vi ska här skapa två klasser, en rack klass, *MotionControl_USB* och en kortklass *MotionControl_USBIO*.

Skapa en Rack klass

I vårt fall så är det kort klassen som kommer att utföra allt jobb och innehålla alla metoder. Rack klassen finns med enbart för att markera rack-nivån, och har inte några metoder eller attribut, vi kommer endast att lägga in ett description attribut i klassen. Vi skapar ett \$ClassHier objekt, och under detta ett \$ClassDef objekt för rack klassen. Objektet namnges *MotionControl_USB* och vi sätter IORack och IO bitarna i attributet Flags.

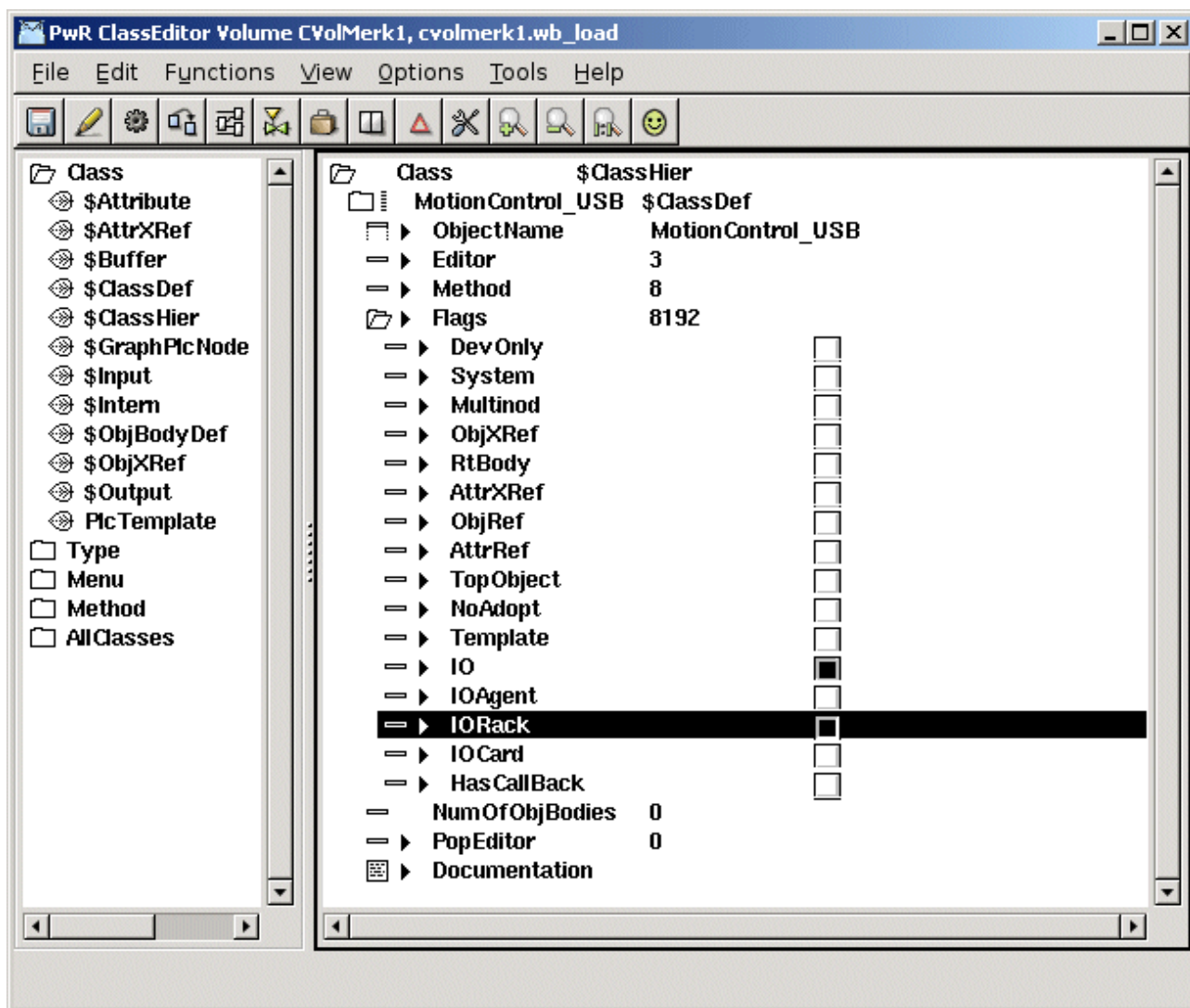


Fig IO och IO Rack bitarna i Flags

Under \$ClassDef objektet definierar vi attributen i klassen. Vi skapar ett \$ObjBodyDef objekt och under detta ett \$Attribute objekt med namnet Description med typen pwrs:Type-\$String80.

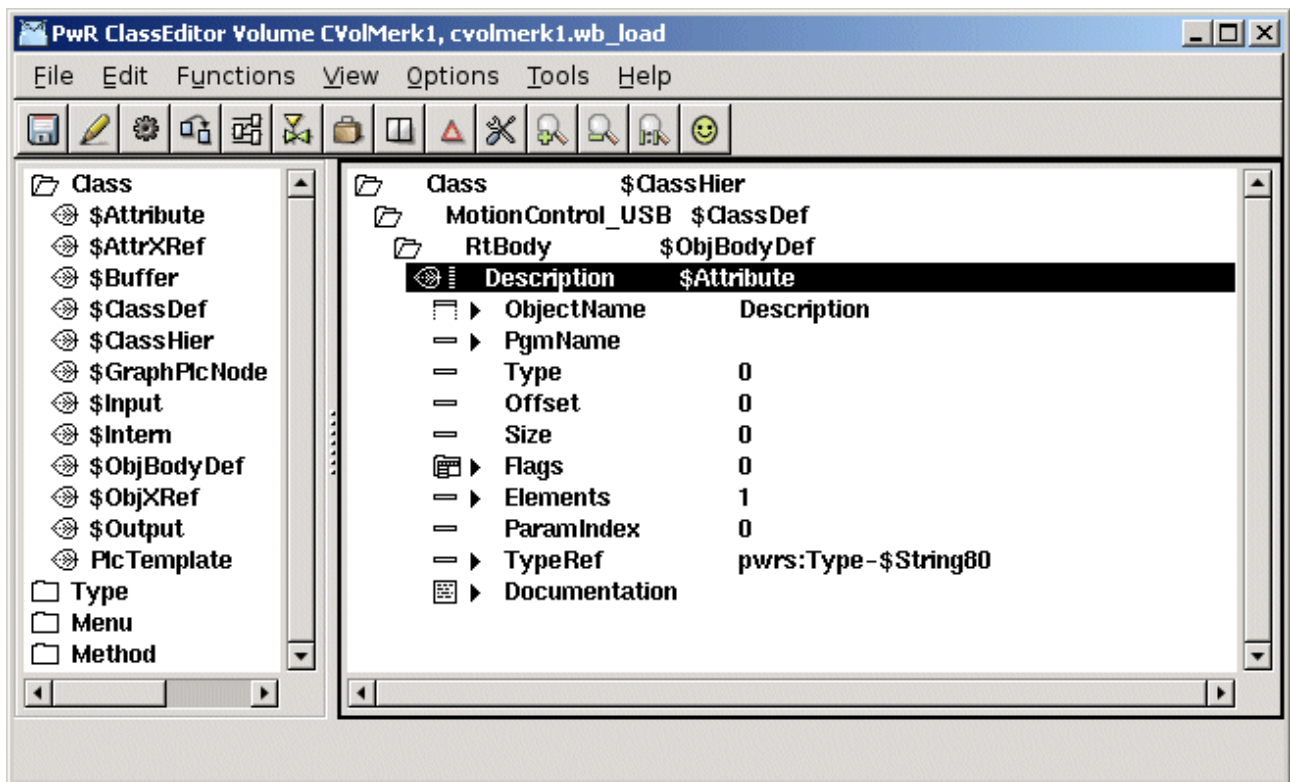


Fig Attribut objekt

Skapa en kortklass

För kortobjekt finns en basklass, *Basecomponent:BaseIOCard*, som man kan utnyttja och som innehåller de vanligaste attributen för ett kort. Vi skapar ytterligare ett \$ClassDef objekt men namnet MotionControl_USBIO, och sätter IO och IOCard bitarna i attributet Flags.

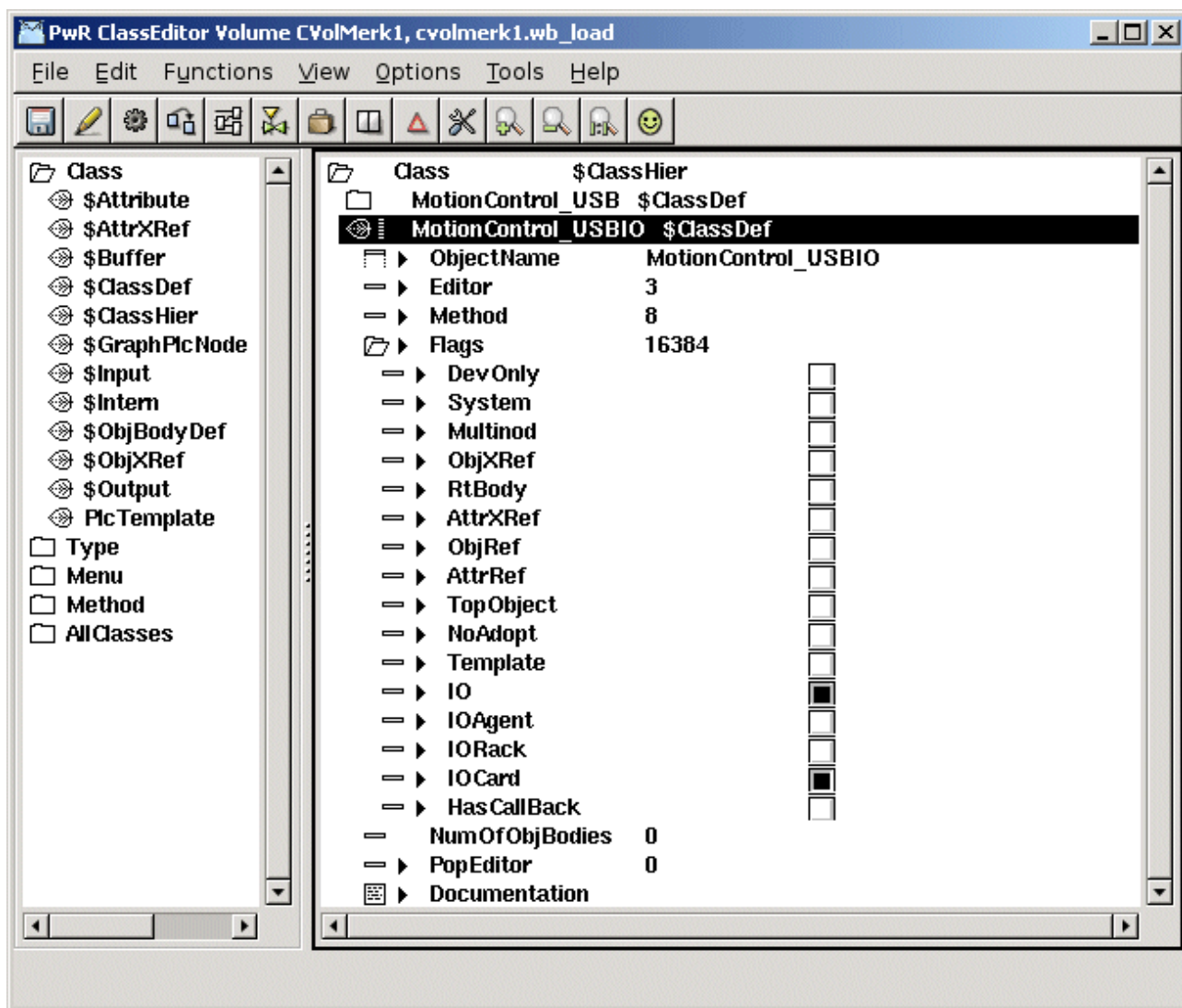


Fig IO och IOCard bitarna i Flags

Vi skapar ett \$ObjBodyDef objekt och ett \$Attribute objekt för att ange att BaseIOCard ska vara en superklass. Attributet namnes Super och som TypeRef sätts Basecomponent:Class-BaseIOCard. Vi kommer nu att ärvä alla attribut och metoder som BaseIOCard innehåller.

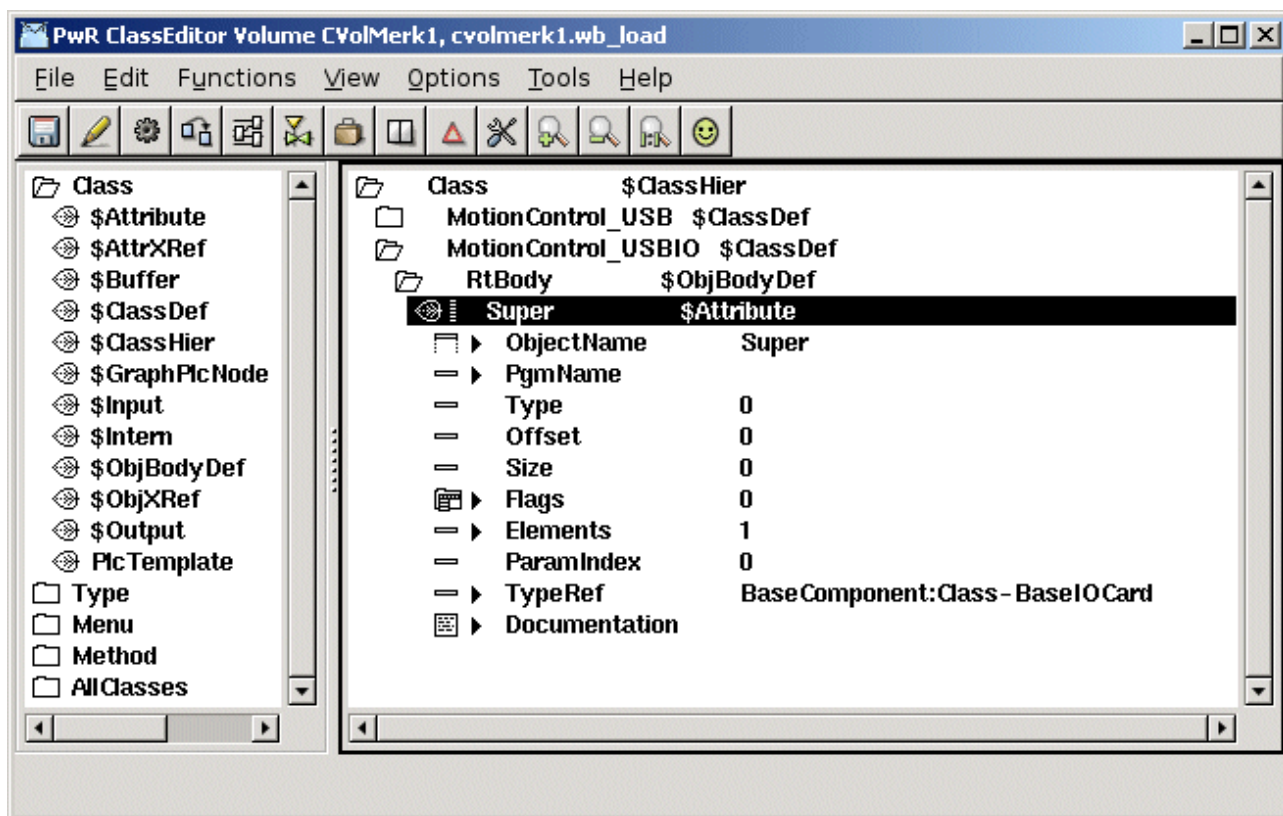


Fig Konfigurering av superklassen BaseIOCard

Vi lägger till ytterligare ett attribut för kortets status, och för statusen skapar vi en uppräkningsstyp, MotionControl_StatusEnum, som innehåller de olika felkoder som status-attributet kan innehålla.

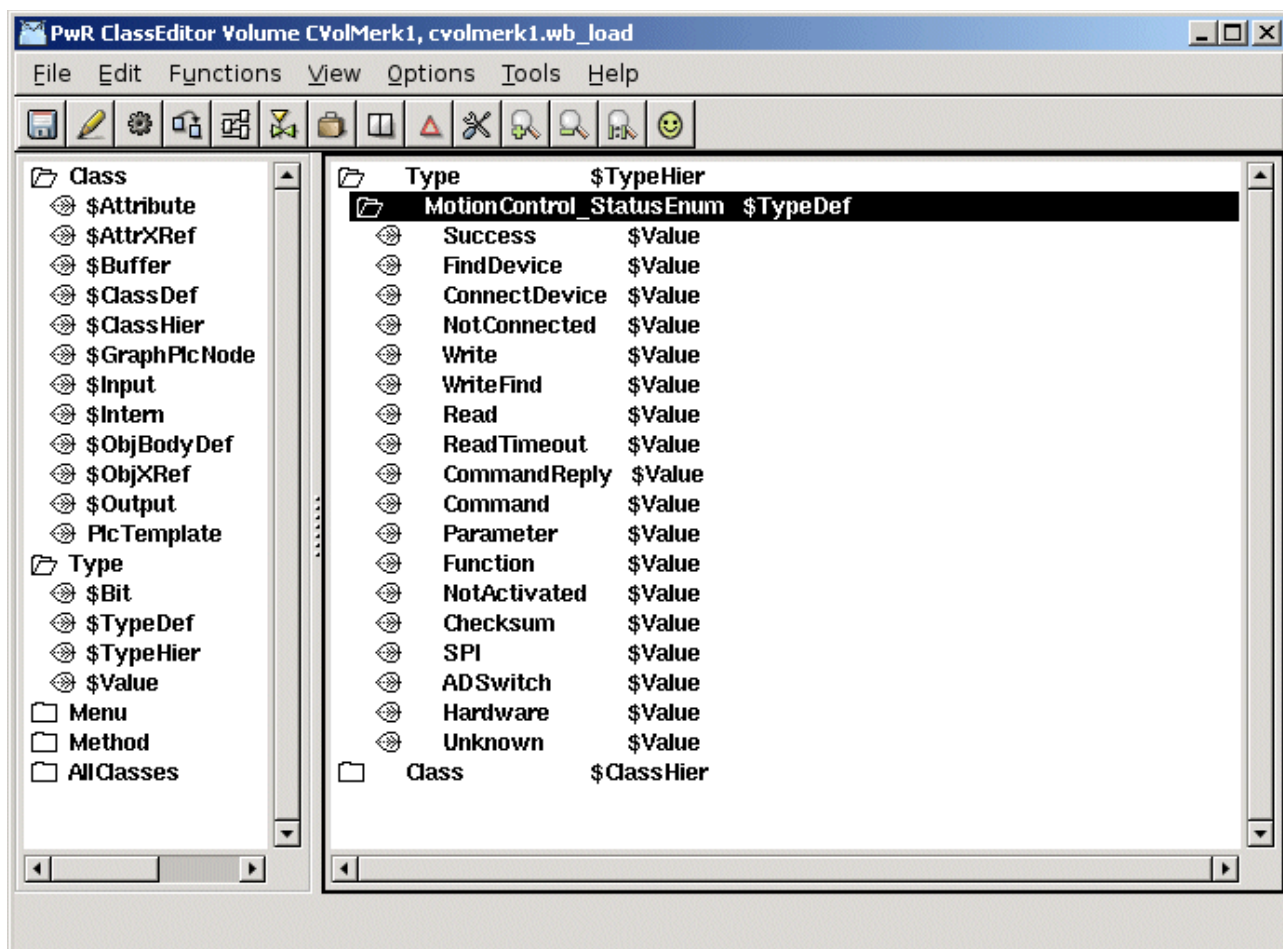


Fig Definition av en enum typ

Status-attributets typ sätts till den skapade statustypen, och i Flags sätts bitarna *State* och *NoEdit*, då attributet inte ska datasättas i konfiguratören utan ges ett värde i runtimemiljön.

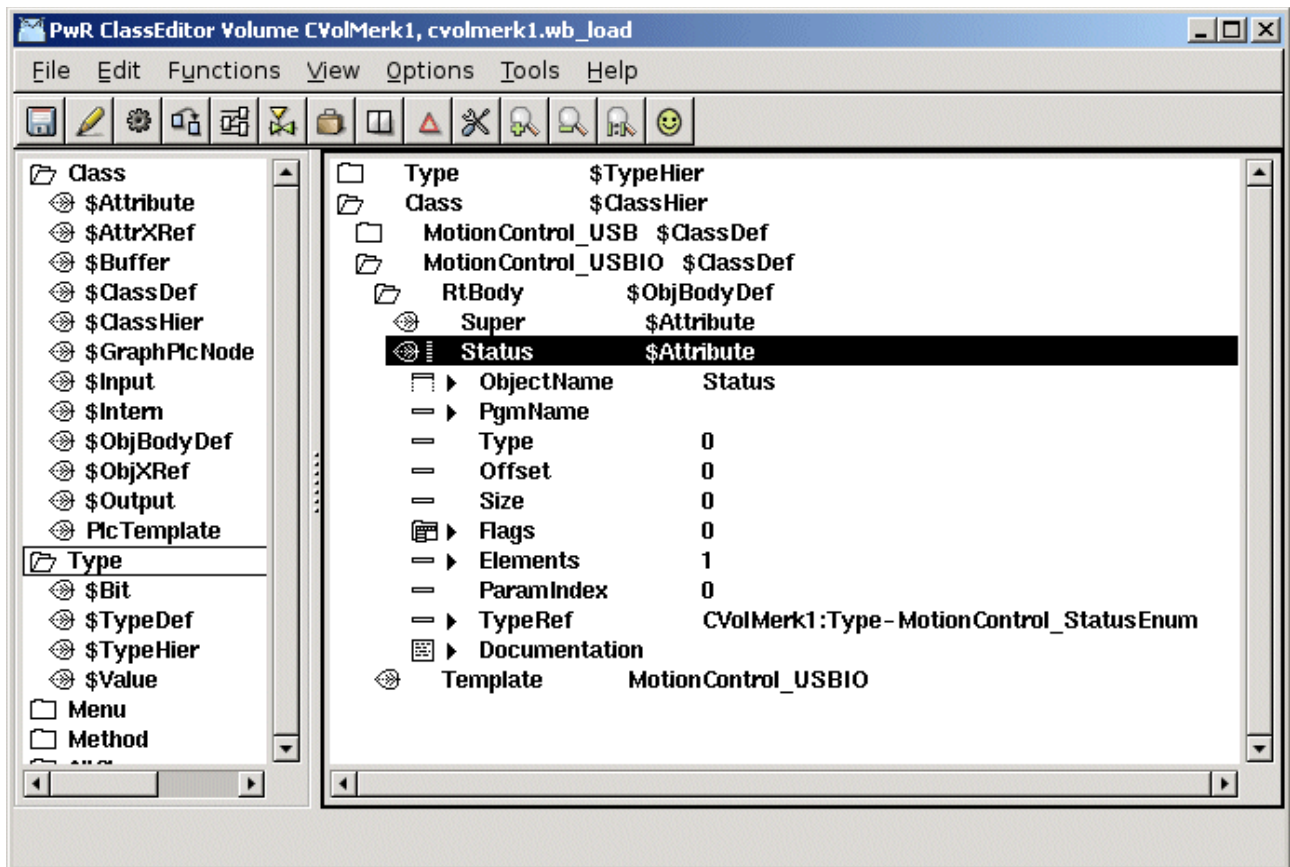


Fig Status attribut av enum typ

Normalt lägger man även in attribut för kanalobjekt i kort klassen, men eftersom USBIO modulen är så flexibel, samma kanal kan konfigureras som en Di, Do eller Ai kanal, kan vi inte lägga kanalobjekten som attribut, utan dessa får konfigureras som enskilda objekt som läggs som barn till kortobjektet i rotvolymen.

USB I/O innehåller en watchdog som återställer enheten om man inte skriver till enheten inom en viss tid. Vi lägger därför till attributet WatchdogTime så att timeout-tiden kan konfigureras.

När man sparar en klass första gången skapas ett Template objekt under \$ClassDef objektet. Detta är en instans av den aktuella klassen som där man kan lägga in defaultvärden på attributen. Vi fyller i Specification, lägger in en URL till databladet, och sätter Process till 1. Vi sätter också MaxNoOfChannels till 21, eftersom det här kortet har 21 kanaler.

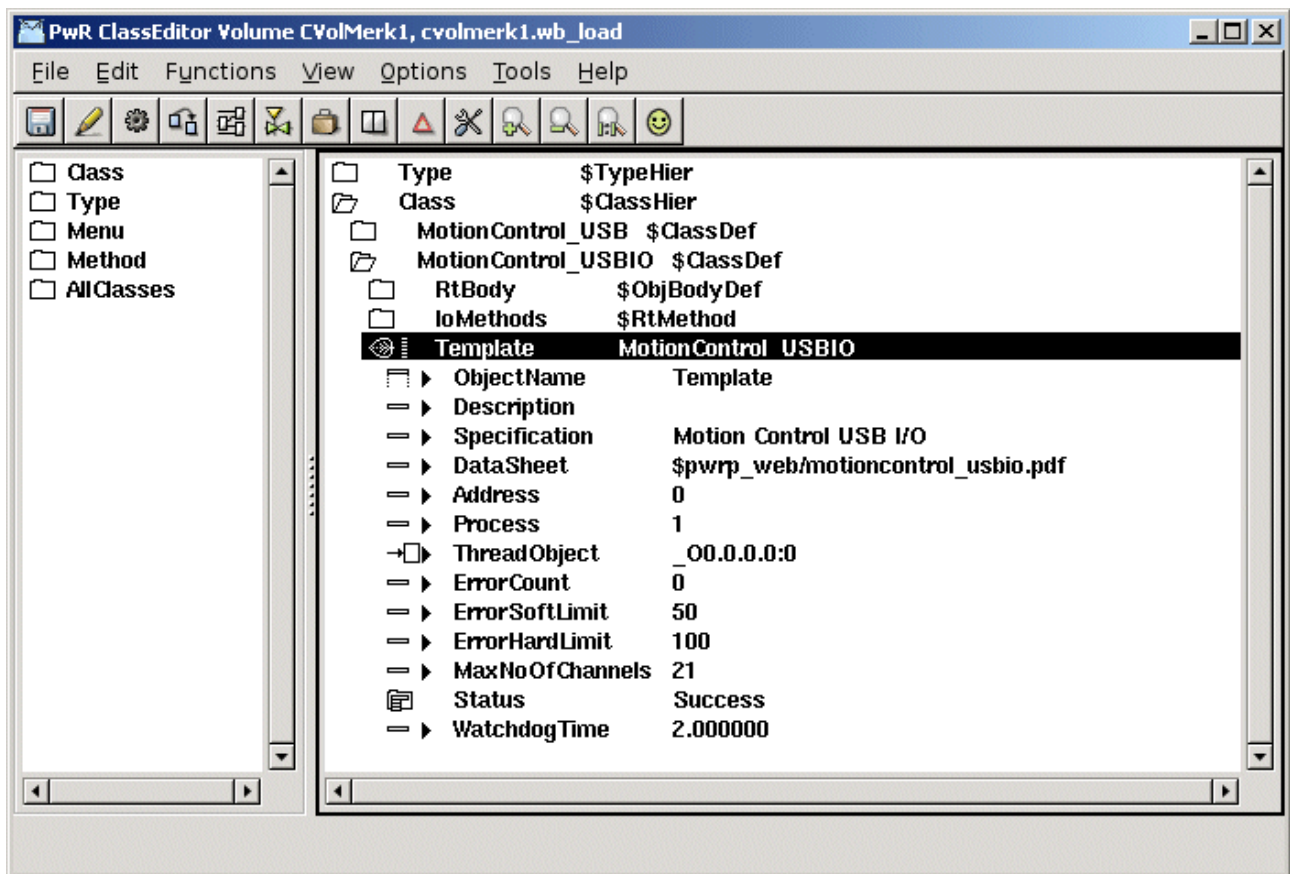


Fig Template objekt

Nästa steg är att lägga in metoderna i klassbeskrivningen. Eftersom kortet innehåller både utgångar och ingångar behöver vi skapa Init, Close, Read och Write metoder. Dessa anges med metodobjekt av type \$Method. Först lägger vi ett \$RtMethod objekt med namnet IoMethods under \$ClassDef objektet. Under detta lägger vi ett \$Method objekt för varje metod. Objektet namnges IoCardInit, IoCardClose, IoCardRead och IoCardWrite. I attributet MethodName lägger vi in den sträng som metoderna kommer att registreras med i c-koden, dvs "MotionControl_USBIO-IoCardInit", etc.

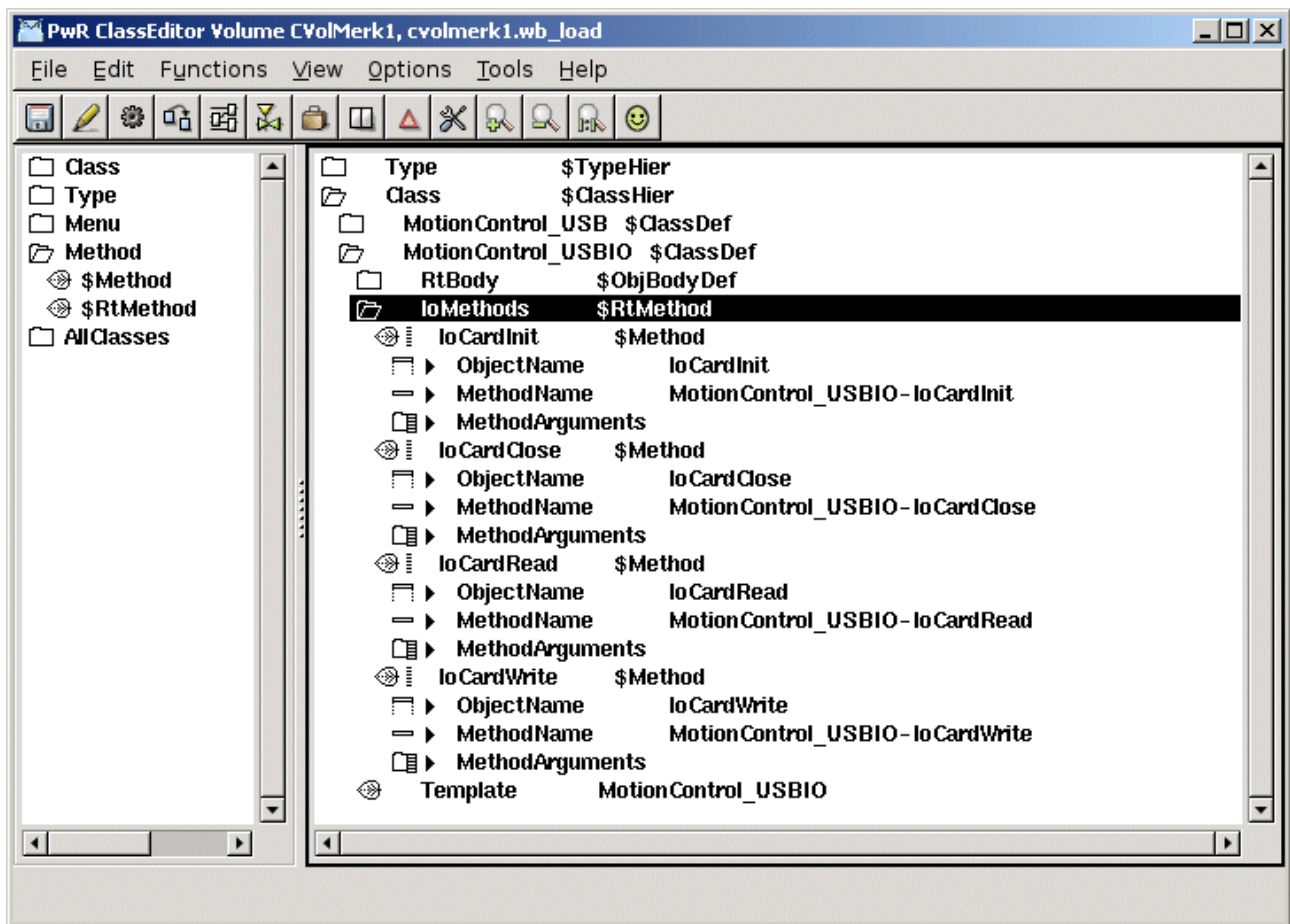


Fig Konfigurering av I/O metoder

Från superklassen BaseIOCard ärver vi en metod att i konfiguratören knyta en objekt till en plctråd.

Bygg klassvolymen

Nu är klasserna skapade och vi sparar, lämnar editering-mod, och skapar laddatafiler för klassvolymen genom att aktivera *Functions/Build Volume* i menyn. Nu skapas också en include-fil med c-struct'ar för klasserna, \$pwrp_inc/pwr_cvolmerk1classes.h.

Installera drivrutin

Ladda hem och packa upp tar-filen för drivrutinen.

```
> tar -xvzf usbio.tar.tz
```

Bygg drivrutinen mha make

```
> cd usbio/driver/linux-2.6
```

```
> make
```

Installera drivrutinen `usbio.ko` som root

```
> insmod usbio.ko
```

Tillåt alla att läsa och skriva till drivrutinen

```
> chmod a+rw /dev/usbio0
```

Till drivrutinen hör ett API med en include-file, `libusbio.h`, och ett arkiv, `libusbio.a`. Include-filen kopieras till `$pwrp_inc`, och arkivet till `$pwrp_lib`.

Skriva metoder

Nästa steg är att skriva c-koden för metoderna.

c-filen `ra_io_m_motioncontrol_usb.c` läggs på `$pwrp_src`.

Eftersom ProviewR är GPL-licensierat måste även koden för metoderna GPL licensieras om programmet distribueras till andra parter. Vi lägger därför in ett GPL huvud i början på filen.

För att förenkla koden låser vi oss till en konfiguration av USB I/O där kanal 0-3 är digitala utgångar, 4-7 digitala ingångar, 8-15 Analoga ingångar, 16 -18 digitala ingångar och 19-20 analoga utgångar.

`ra_io_m_motioncontrol_usb.c`

```
/*
 * ProviewR   $Id$
 * Copyright (C) 2005 SSAB Oxelösund.
 *
 * This program is free software; you can redistribute it and/or
 * modify it under the terms of the GNU General Public License as
 * published by the Free Software Foundation, either version 2 of
 * the License, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with the program, if not, write to the Free Software
 * Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
 */

#include "pwr.h"
#include "pwr_basecomponentclasses.h"
#include "pwr_cvolmerklclasses.h"
#include "rt_io_base.h"
#include "rt_io_card_init.h"
#include "rt_io_card_close.h"
#include "rt_io_card_read.h"
#include "rt_io_card_write.h"
#include "rt_io_msg.h"
#include "libusbio.h"

typedef struct {
    int USB_Handle;
} io_sLocal;

// Init method
static pwr_tStatus IoCardInit( io_tCtx ctx,
                               io_sAgent *ap,
                               io_sRack *rp,
                               io_sCard *cp)
{
    int i;
    int timeout;
    io_sLocal *local;
    pwr_sClass_MotionControl_USBIO *op = (pwr_sClass_MotionControl_USBIO *)cp->op;

    local = (io_sLocal *) calloc( 1, sizeof(io_sLocal));
    cp->Local = local;
}
```

```

// Configure 4 Do and 4 Di on Port A
op->Status = USBIO_ConfigDIO( &local->USB_Handle, 1, 240);
if ( op->Status)
    errh_Error( "IO Init Card '%s', Status %d", cp->Name, op->Status);

// Configure 8 Ai on Port B
op->Status = USBIO_ConfigAI( &local->USB_Handle, 8);
if ( op->Status)
    errh_Error( "IO Init Card '%s', Status %d", cp->Name, op->Status);

// Configure 3 Di and 2 Ao on Port C
op->Status = USBIO_ConfigDIO( &local->USB_Handle, 3, 7);
if ( op->Status)
    errh_Error( "IO Init Card '%s', Status %d", cp->Name, op->Status);
op->Status = USBIO_ConfigAO( &local->USB_Handle, 3);
if ( op->Status)
    errh_Error( "IO Init Card '%s', Status %d", cp->Name, op->Status);

// Calculate conversion coefficients for Ai
for ( i = 8; i < 16; i++) {
    if ( cp->chanlist[i].cop &&
        cp->chanlist[i].sop &&
        cp->chanlist[i].ChanClass == pwr_cClass_ChanaI)
        io_AiRangeToCoef( &cp->chanlist[i]);
}

// Calculate conversion coefficients for Ao
for ( i = 19; i < 21; i++) {
    if ( cp->chanlist[i].cop &&
        cp->chanlist[i].sop &&
        cp->chanlist[i].ChanClass == pwr_cClass_Chanao)
        io_AoRangeToCoef( &cp->chanlist[i]);
}

// Configure Watchdog
timeout = 1000 * op->WatchdogTime;
op->Status = USBIO_ConfigWatchdog( &local->USB_Handle, 1, timeout, 1,
                                   port_mask, port, 3);

errh_Info( "Init of USBIO card '%s'", cp->Name);

return IO__SUCCESS;
}

// Close method
static pwr_tStatus IoCardClose( io_tCtx ctx,
                                io_sAgent *ap,
                                io_sRack *rp,
                                io_sCard *cp)
{
    free( cp->Local);
    return IO__SUCCESS;
}

// Read Method
static pwr_tStatus IoCardRead( io_tCtx ctx,
                                io_sAgent *ap,
                                io_sRack *rp,
                                io_sCard *cp)
{

```

```

io_sLocal *local = cp->Local;
pwr_sClass_MotionControl_USBIO *op = (pwr_sClass_MotionControl_USBIO *)cp->op;
int value = 0;
int i;
unsigned int m;
pwr_tUInt32 error_count = op->Super.ErrorCount;

// Read Di on channel 4 - 8
op->Status = USBIO_ReadDI( &local->USB_Handle, 1, &value);
if ( op->Status)
    op->Super.ErrorCount++;
else {
    // Set Di value in area object
    m = 1 << 4;
    for ( i = 4; i < 8; i++) {
        *(pwr_tBoolean *)cp->chanlist[i].vbp = ((value & m) != 0);
        m = m << 1;
    }
}

// Read Ai on channel 8 - 16
for ( i = 0; i < 8; i++) {
    io_sChannel *chanp = &cp->chanlist[i + 8];
    pwr_sClass_ChanaAi *cop = (pwr_sClass_ChanaAi *)chanp->cop;
    pwr_sClass_Ai *sop = (pwr_sClass_Ai *)chanp->sop;

    if ( cop->CalculateNewCoef)
        // Request to calculate new coefficients
        io_AiRangeToCoef( chanp);

    op->Status = USBIO_ReadADVal( &local->USB_Handle, i + 1, &ivalue);
    if ( op->Status)
        op->Super.ErrorCount++;
    else {
        io_ConvertAi( cop, ivalue, &actvalue);

        // Filter the Ai value
        if ( sop->FilterType == 1 &&
            sop->FilterAttribute[0] > 0 &&
            sop->FilterAttribute[0] > ctx->ScanTime) {
            actvalue = *(pwr_tFloat32 *)chanp->vbp +
                ctx->ScanTime / sop->FilterAttribute[0] *
                (actvalue - *(pwr_tFloat32 *)chanp->vbp);
        }

        // Set value in area object
        *(pwr_tFloat32 *)chanp->vbp = actvalue;
        sop->SigValue = cop->SigValPolyCoef1 * ivalue + cop->SigValPolyCoef0;
        sop->RawValue = ivalue;
    }
}

// Check Error Soft and Hard Limit

// Write warning message if soft limit is reached
if ( op->Super.ErrorCount >= op->Super.ErrorSoftLimit &&
    error_count < op->Super.ErrorSoftLimit)
    errh_Warning( "IO Card ErrorSoftLimit reached, '%s'", cp->Name);

// Stop I/O if hard limit is reached
if ( op->Super.ErrorCount >= op->Super.ErrorHardLimit) {
    errh_Error( "IO Card ErrorHardLimit reached '%s', IO stopped", cp->Name);
}

```

```

        ctx->Node->EmergBreakTrue = 1;
        return IO__ERRDEVICE;
    }
    return IO__SUCCESS;
}

// Write method
static pwr_tStatus IoCardWrite( io_tCtx ctx,
                                io_sAgent *ap,
                                io_sRack *rp,
                                io_sCard *cp)
{
    io_sLocal *local = cp->Local;
    pwr_sClass_MotionControl_USBIO *op = (pwr_sClass_MotionControl_USBIO *)cp->op;
    int value = 0;
    float fvalue;
    int i;
    unsigned int m;
    pwr_tUInt32 error_count = op->Super.ErrorCount;
    pwr_sClass_Chao *cop;

    // Write Do on channel 1 - 4
    m = 1;
    value = 0;
    for ( i = 0; i < 4; i++) {
        if ( *(pwr_tBoolean *)cp->chanlist[i].vbp)
            value |= m;
        m = m << 1;
    }
    op->Status = USBIO_WriteD0( &local->USB_Handle, 1, value);
    if ( op->Status) op->Super.ErrorCount++;

    // Write Ao on channel 19 and 20
    if ( cp->chanlist[19].cop &&
        cp->chanlist[19].sop &&
        cp->chanlist[19].ChanClass == pwr_cClass_Chao) {
        cop = (pwr_sClass_Chao *)cp->chanlist[19].cop;

        if ( cop->CalculateNewCoef)
            // Request to calculate new coefficients
            io_AoRangeToCoef( &cp->chanlist[19]);

        fvalue = *(pwr_tFloat32 *)cp->chanlist[19].vbp * cop->OutPolyCoef1 +
            cop->OutPolyCoef0;
        op->Status = USBIO_WriteA0( &local->USB_Handle, 1, fvalue);
        if ( op->Status) op->Super.ErrorCount++;
    }

    if ( cp->chanlist[20].cop &&
        cp->chanlist[20].sop &&
        cp->chanlist[20].ChanClass == pwr_cClass_Chao) {
        cop = (pwr_sClass_Chao *)cp->chanlist[20].cop;

        if ( cop->CalculateNewCoef)
            // Request to calculate new coefficients
            io_AoRangeToCoef( &cp->chanlist[20]);

        fvalue = *(pwr_tFloat32 *)cp->chanlist[20].vbp * cop->OutPolyCoef1 +
            cop->OutPolyCoef0;
        op->Status = USBIO_WriteA0( &local->USB_Handle, 2, fvalue);
    }
}

```



```

    if ( op->Status) op->Super.ErrorCount++;
}

// Check Error Soft and Hard Limit

// Write warning message if soft limit is reached
if ( op->Super.ErrorCount >= op->Super.ErrorSoftLimit &&
    error_count < op->Super.ErrorSoftLimit)
    errh_Warning( "IO Card ErrorSoftLimit reached, '%s'", cp->Name);

// Stop I/O if hard limit is reached
if ( op->Super.ErrorCount >= op->Super.ErrorHardLimit) {
    errh_Error( "IO Card ErrorHardLimit reached '%s', IO stopped", cp->Name);
    ctx->Node->EmergBreakTrue = 1;
    return IO__ERRDEVICE;
}

return IO__SUCCESS;
}

// Every method should be registred here

pwr_dExport pwr_BindIoUserMethods(MotionControl_USBIO) = {
    pwr_BindIoUserMethod(IoCardInit),
    pwr_BindIoUserMethod(IoCardClose),
    pwr_BindIoUserMethod(IoCardRead),
    pwr_BindIoUserMethod(IoCardWrite),
    pwr_NullMethod
};

```

Registrera klassen

För att I/O ramverket ska kunna hitta metoderna för klassen måste dessa registreras. Det gör man genom att skapa filen `$pwrp_src/rt_io_user.c`. Här använder man macrona `pwr_BindIoUserMethods` och `pwr_BindIoUserClass` för varje klass som har metoder.

rt_io_user.c

```

#include "pwr.h"
#include "rt_io_base.h"

pwr_dImport pwr_BindIoUserMethods(MotionControl_USBIO);

pwr_BindIoUserClasses(User) = {
    pwr_BindIoUserClass(MotionControl_USBIO),
    pwr_NullClass
};

```

Makefile

För att kompilera c-filerna skapar vi en make-file på `$pwrp_src`, `$pwrp_src/makefile`. Denna kompilerar `ra_io_m_motioncontro_usbio.c` och `rt_io_user.c`, och lägger in objektsmodulerna på katalogen `$pwrp_obj`.

makefile

```

mars2_top : mars2

include $(pwr_exe)/pwrp_rules.mk

mars2_modules = $(pwrp_obj)/ra_io_m_motioncontrol_usbio.o \
                $(pwrp_obj)/rt_io_user.o

```

```
# Main rule
mars2 : $(mars2_modules)
        @ echo "***** Mars2 modules built *****"

# Modules

$(pwrp_obj)/ra_io_m_motioncontrol_usbio.o : \
    $(pwrp_src)/ra_io_m_motioncontrol_usbio.c \
    $(pwrp_inc)/pwr_cvolmerk1classes.h

$(pwrp_obj)/rt_io_user.o : $(pwrp_src)/rt_io_user.c
```

Länkfil

Vi väljer att anropa metoderna från plc processen, och måste då länka plcprogrammet med objektsmodulerna för metoderna. Det gör vi genom att lägga en länkfil på \$pwrp_exe. Vi måste även länka med arkivet för API:t mot drivrutinen för USB I/O, `libusbio.a`. Namnet på länkfilen innehåller nodnamn och buss.

plc_mars2_0507.opt

```
$pwrp_obj/rt_io_user.o $pwrp_obj/ra_io_m_motioncontrol_usbio.o -lpwr_rt \
    -lusbio -lpwr_usb_dummy -lpwr_pnak_dummy
```

Konfigurera nodhierarkin

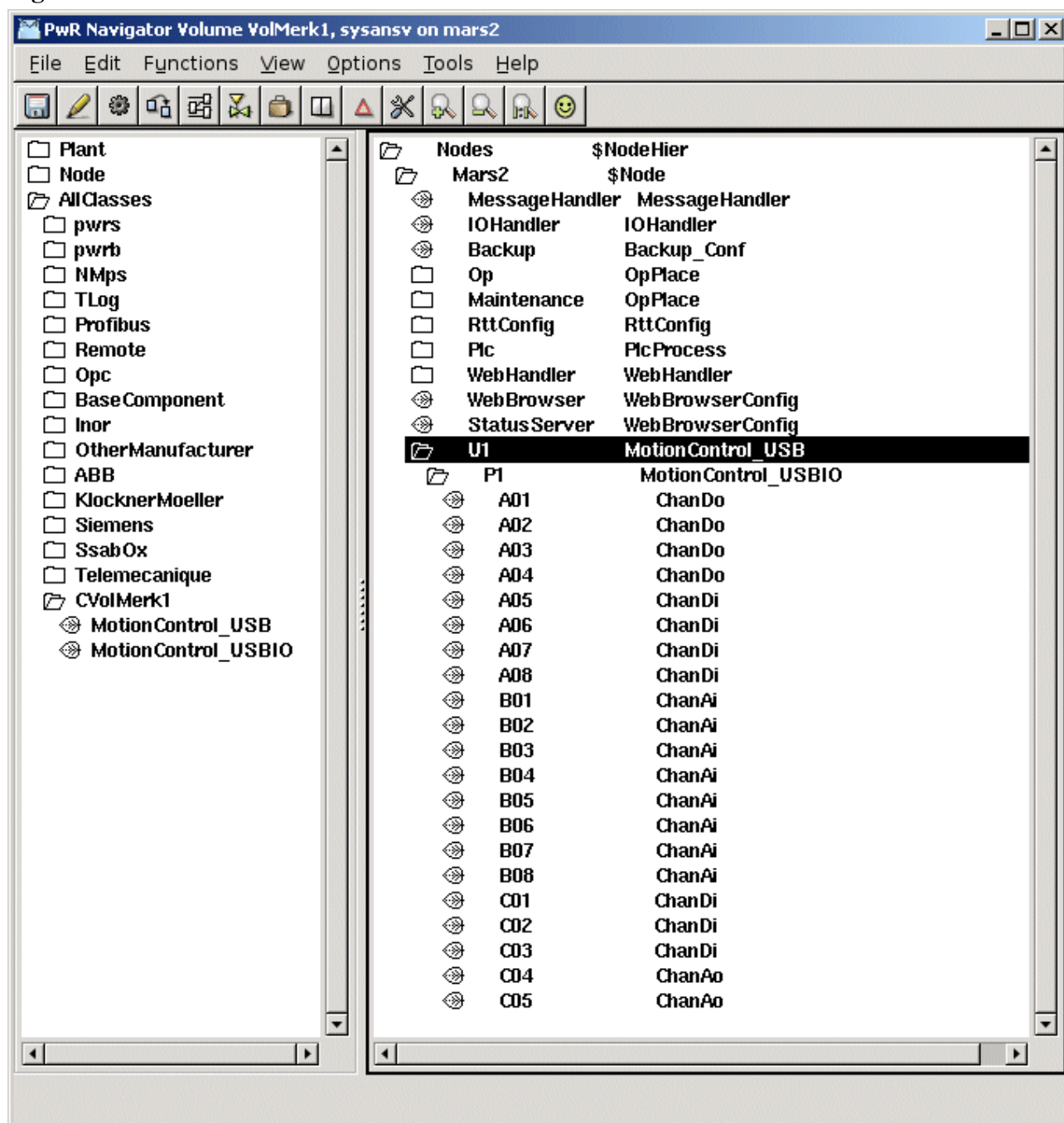
Nu är det dags att konfigurera I/O objekten i nodehierarkin med objekt av de klasser som vi har skapat.

Rotvolymen i vår projekt heter VolMerk1 och vi öppnar konfiguratören med

```
> pwrsvolmerk1
```

I paletten till vänster, under mappen AllClasses hittar vi klassvolymen i projektet, och under denna de två klasser för USB I/O som vi har skapat. Under \$Node-objektet lägger vi ett rack-objekt av klassen MotionControl_USB, och under detta ett kort-objekt av klassen MotionControl_USBIO. Eftersom kanalobjekten inte ligger internt i kortobjektet, måste vi skapa kanalobjekt för de kanaler vi tänker använda under kort-objektet. Se resultatet i Fig Nodehierarkin.

Fig Nodehierarkin



Vi sätter attributet Number, som markerar index i kanallistan, till 0 för det första kanalobjektet, 1 för det andra etc. Vi sätter Process till 1 i rack och kort objektet, och knyter även dessa till en plc-tråd, genom att välja ut ett PlcThread objekt, och aktivera Connect PlcThread i popupmenyn för rack och kort objektet.

För analoga kanaler måste även anges områden för konvertering till ingenjörstorhet. Råvärdesområdet för Ai-kanaler är 0-1023, och signalområdet 0-5 V. Vi konfigurerar kanalerna för ett ingenjörstorhet på 0-100 enligt *Fig Ai kanal*.

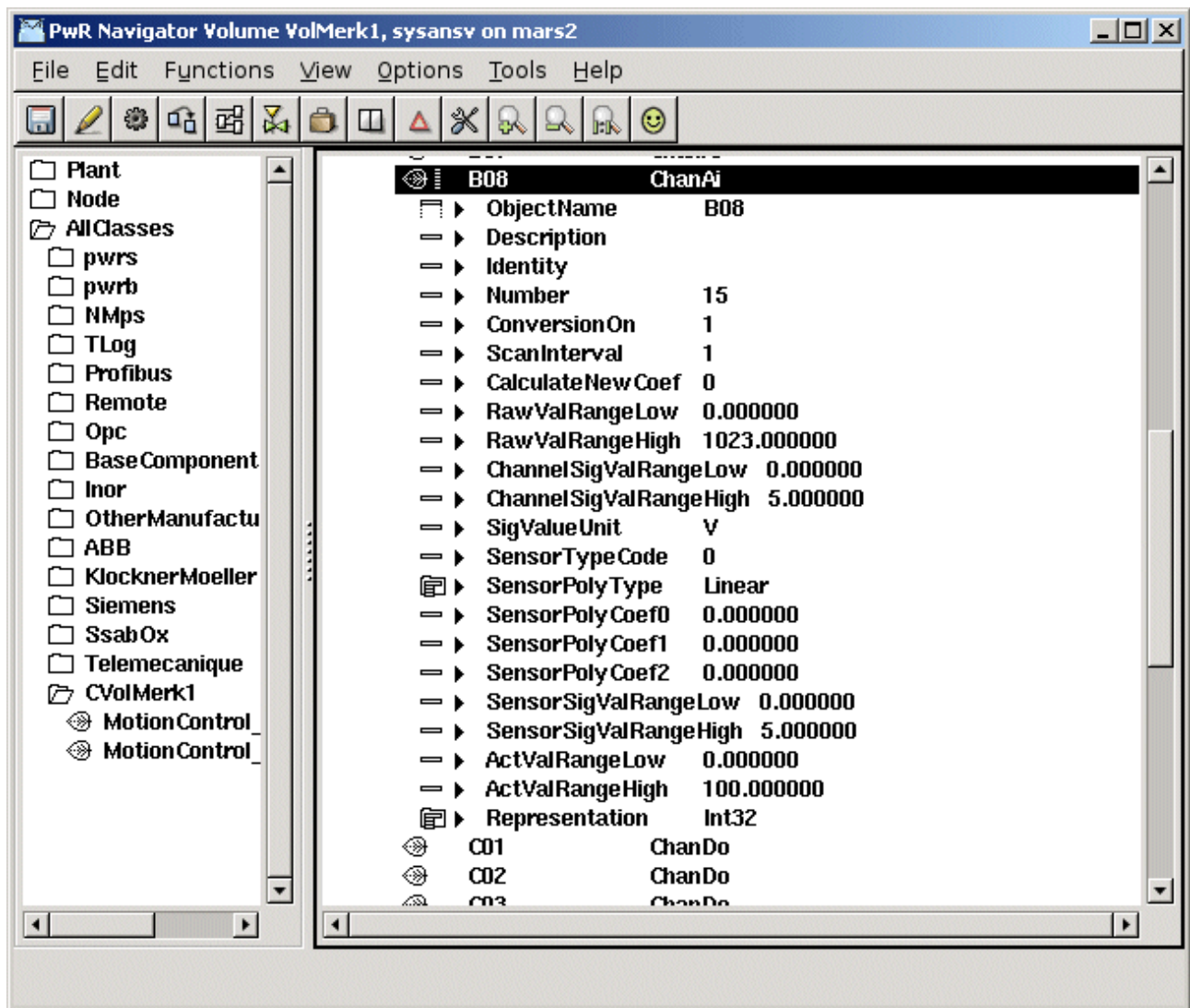


Fig Ai kanal

För Ao-kanalerna får man signalvärdet vid läsningen, och vi konfigurerar med ett råvärdesområde på 0-5 V enligt *Fig Ao kanal*.

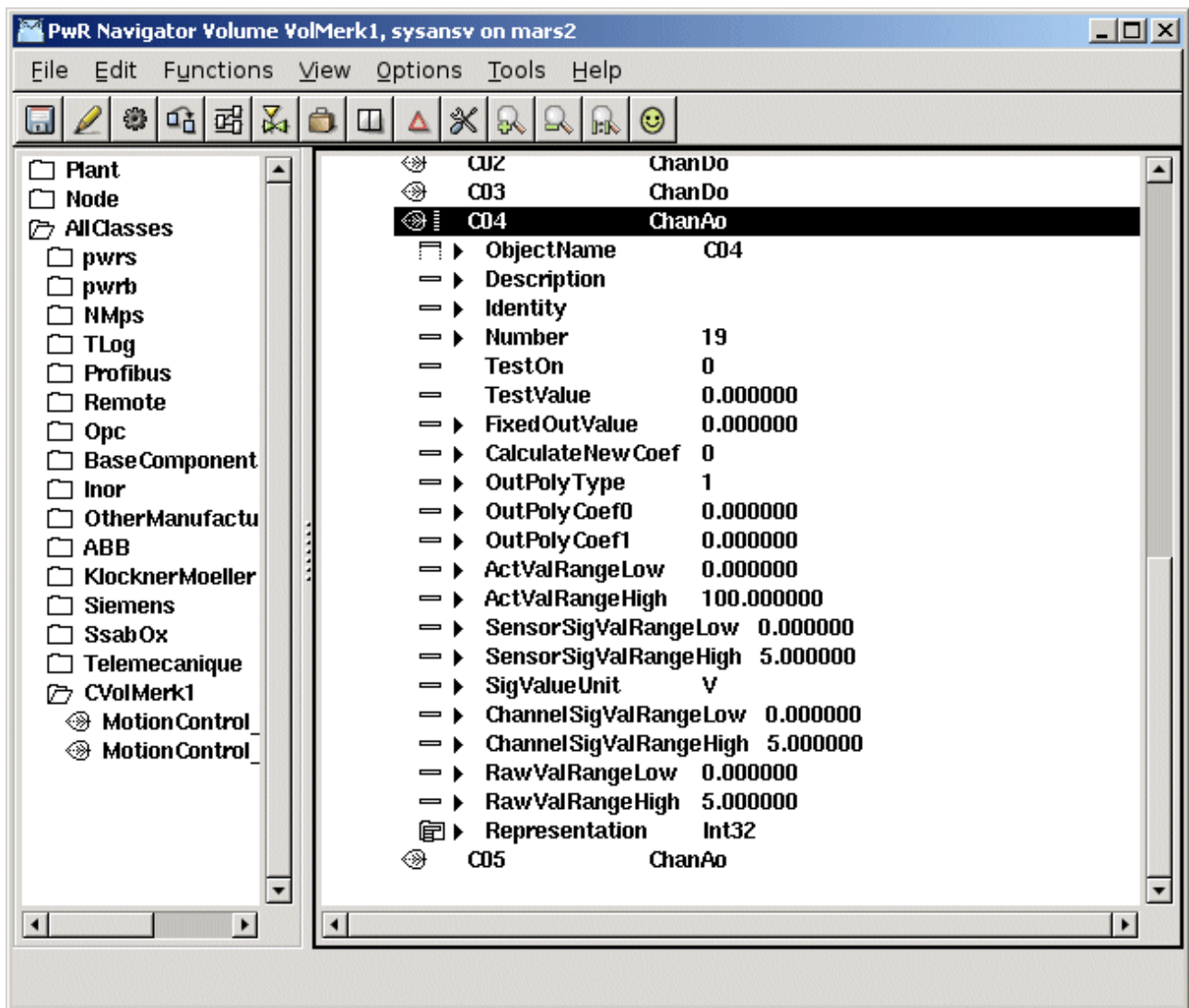


Fig Ao kanal

Vi måste även skapa signaler-objekt i anläggningshierarin, av typen Di, Do, Ai och Ao, och koppla dessa till respektive kanal. Det måste även finnas ett PlcPgm för att ett plc program ska skapas som hanterar I/O't. Nu återstår bara att bygga noden, distribuera, kontrollera strappningen av USB I/O kortet, koppla in det på USB porten och starta ProviewR.

Lägga in i ProviewR's bassystem

Klassvolym

Först måste man bestämma sig för i vilken klassvolym I/O systemet ska ligga i. Mindre I/O system läggs i volymen OtherIO, men om det är ett stort I/O system kan man överväga att skapa en egen klassvolym och en egen modul för I/O systemet.

OtherIO

OtherIO är en modul avsedd för mindre I/O system. Modulen innehåller en klassvolym som ligger på wbl/mcomp/src, moduler som läggs in i arkivet libpwr_rt.a på lib/rt/src, objektsbilder på /mmi/mcomp/src importerade h-filer och arkiv för gränssnitt på exp/rt/src.

Man knyter upp sig till modulen med

```
> pwre module otherio
```

Skapa klasser

Gå till katalogen wbl/mcom/src och starta klassesitorn för klassvolymen OtherIO som ligger i filen otherio.wb_load.

```
> cd $pwre_sroot/wbl/mcomp/src
```

```
> wblstart.sh otherio.wb_load
```

Lägg in klasser för rack och kort på samma sätt som när man lägger in klasserna i ett projekt. Observera att man inte ska bygga volymen från klassesitorn. När editeringen är klar går man ur klassesitorn och bygger med följande kommandon.

Skapa include-filer för c och c++ med

```
> pwre build wbl mcomp src copy
```

Skapa en ny dbs-fil med

```
> pwre build wbl mcomp src lib
```

Skapa dokumentation med

```
> pwre build wbl mcomp src exe
```

Skapa metoder

Metoderna läggs på lib/rt/src/os_linux och filerna ska ha prefixet rt_io_m_. Metoderna skrivs på samma sätt som i ett projekt beskrivet ovan. Metoderna registreras med macrona pwr_BindIoMethods och pwr_BindIoMethod.

```
pwr_dExport pwr_BindIoMethods(MotionControl_USBI0) = {  
    pwr_BindIoMethod(IoCardInit),  
    pwr_BindIoMethod(IoCardClose),  
    pwr_BindIoMethod(IoCardRead),  
    pwr_BindIoMethod(IoCardWrite),  
    pwr_NullMethod  
};
```

c-filerna kompileras med kommandot

```
> pwre build lib rt src
```

Registrera klassen

Alla klasser som har I/O metoder måste registreras. Det görs i filen lib/rt/src/rt_io_otherio.meth, som innehåller namnet på klasserna, en klass på varje rad.

Objektbilder

Objektbilderna läggs på mmi/rt/src och ska ha samma namn som klassen med små bokstäver, och med prefixet pwr_c_. Ge startas med gestart.sh.

```
> cd $pwre_sroot/mmi/mcomp/src
```

```
> gestart.sh pwr_c_'classname'
```

Bygg bilderna med

```
> pwr build mmi mcomp src
```

Merge

När allt i modulen har byggts ska den mergas med övriga moduler.

```
> pwre merge
```

Separat Modul

Todo...

Om Hilscher Profibus agenten

Agenten länkar mot Hilschers API. Detta kräver att en kernelmodul laddas, vilken hanterar kommunikationen med kortet. APIet kommunicerar sedan med kernelmodulen via /dev/cif.

Det är huvudsakligen två manualer som är relevanta vid arbete med Hilscher agenten. Den ena finns i tarbollen för Hilschers Linux-drivrutin och heter CIFLinux_en.pdf, denna beskriver generella aspekter av Linux versionen av APIet. Den andra manualen heter dpm_pie.pdf och återfinns på den CD skiva som medföljer kortet. Manualen behandlar Profibus DP funktionaliteten hos kortet. En annan manual som bör nämnas är TKE.PDF (även denna finns på drivrutins Cdn), denna beskriver vissa detaljer om hårdvaran som inte de övriga manualerna går in på.

Agentmodulen innehåller all kortspezifisk kod. Första delen innehåller hjälpfunktioner för att använda APIet. Den andra delen innehåller metoderna som exporteras till ProviewR.